
task-app 30日間カリキュラム

Day 07

認証バックエンドを作ってログインを動かそう

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	3
今日のゴール	4
なぜこれを作るのか	5
実装ステップ一覧	8
Step 0: 控えを取り、配布済みのDB列を確認し、共通部品を書き直す(読む目安: 8分)	10
Step 1: session.ts を作り直す(JWT セッション管理・読む目安: 12分)	16
Step 2: trpc.ts を作り直す(API の土台・読む目安: 10分)	30
Step 3: auth.ts を作り直す(認証ルーター・読む目安: 15分)	39
Step 4: API を繋ぎ直す(ルーター登録 + HTTP ハンドラー・読む目安: 5分)	65
Step 5: middleware.ts を作る(ルート保護・読む目安: 8分)	68
Step 6: ログインして動作確認する(読む目安: 5分)	81
Step 7: DevTools で JWT と Cookie を確認する(読む目安: 5分)	85
Pro パターンで書こう(認証ガードは early return で道順を見せる)	90
完成コード全体	92
今日のまとめ	138
つまずきポイント	139
今日学んだ用語	142
応用課題: 認証失敗の表示を 2 通りで確かめる	143
理解チェック	144
次回予告	145
次に読むもの	145
奥付	146

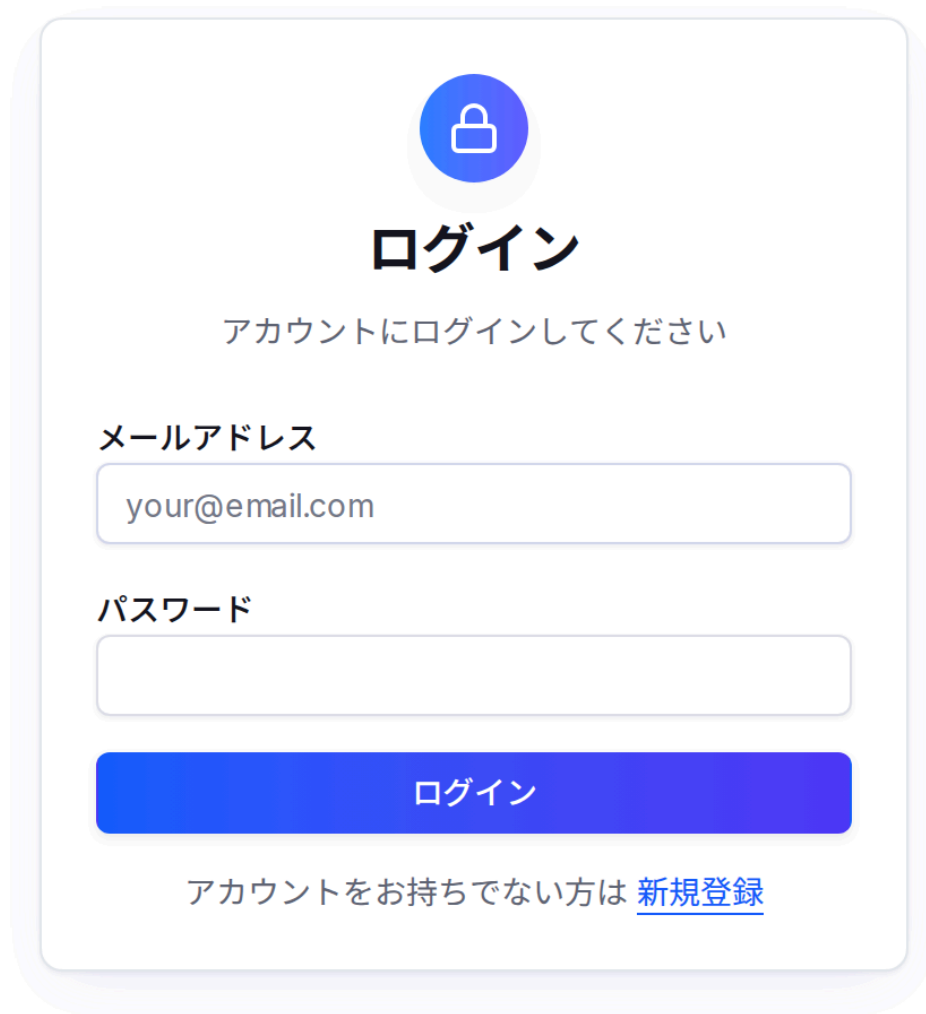
A login form with a light blue background and rounded corners. At the top center is a blue circular icon with a white padlock. Below it is the title 'ログイン' in bold black text, followed by the instruction 'アカウントにログインしてください' in a smaller font. There are two input fields: the first is labeled 'メールアドレス' and contains the text 'your@email.com'; the second is labeled 'パスワード' and is empty. Below the inputs is a large blue button with the text 'ログイン' in white. At the bottom, there is a link '新規登録' in blue text, preceded by the text 'アカウントをお持ちでない方は'.

図 1: ログイン画面。メールアドレスとパスワードの入力欄、ログインボタン、登録リンクが並んでいる

前回の振り返り

Day 05-06 でログイン画面と登録画面の UI を作りました。画面の動作確認に使える認証バックエンドは配布スターターに入っています。

今日はその「裏側」を自分の手で作り直します。完成済みコードを残したまま読むのではなく、対象ファイルの中身を空にし、小さな部品から順番につなぎ直します。

今日のゴール

認証処理を自分で書き直し、ログイン・登録が動くことを確認します。その過程で、JWT トークン・bcrypt パスワード検証・HttpOnly Cookie・tRPC の仕組みを体験的に学びます。

- ☐ `src/lib/session.ts` — JWT セッション管理を作り直します。
- ☐ `prisma/schema.prisma` — scaffold が配布したセッション版数の列を確認します。
- ☐ `src/lib/password.ts` — 配布済みファイルを書き直し、bcrypt 用のパスワード条件を共通化します。
- ☐ `src/server/api/trpc.ts` — tRPC の土台を作り直します。
- ☐ `src/server/api/routers/auth.ts` — 認証 API を作り直します。
- ☐ `src/server/api/routers/_helpers/select.ts` — 返すユーザー情報の項目を定義します。
- ☐ `src/server/api/routers/_helpers/user-email-conflict.ts` — 配布済みファイルを書き直し、メール重複エラーを見分けます。
- ☐ `src/server/api/root.ts` — ルーターの束ね方を作り直します。
- ☐ `src/app/api/trpc/[trpc]/route.ts` — HTTP ハンドラーを作り直します。
- ☐ `src/middleware.ts` — ルート保護を作ります。
- ☐ DevTools でログインの流れを確認します。

なぜこれを作るのか

Day 05 ではブラウザに表示するログイン画面(フロントエンド)を作りました。メールアドレスとパスワードの照合には、配布スターターに入っているサーバー側の認証処理を使って動作確認しました。今日はその認証処理を自分で書き直し、データベースとの照合から Cookie の保存までをつなぎます。

今日は、ログインの「裏方」に加えてセッション版数とパスワード検証の共通部品も作ります。

例え話: レストランに例えると Day 05-06 で作ったのは注文用紙(フォーム)。今日は厨房(サーバー)と配膳システム(API)を作って注文がちゃんと通るようにします。

認証フローの全体像

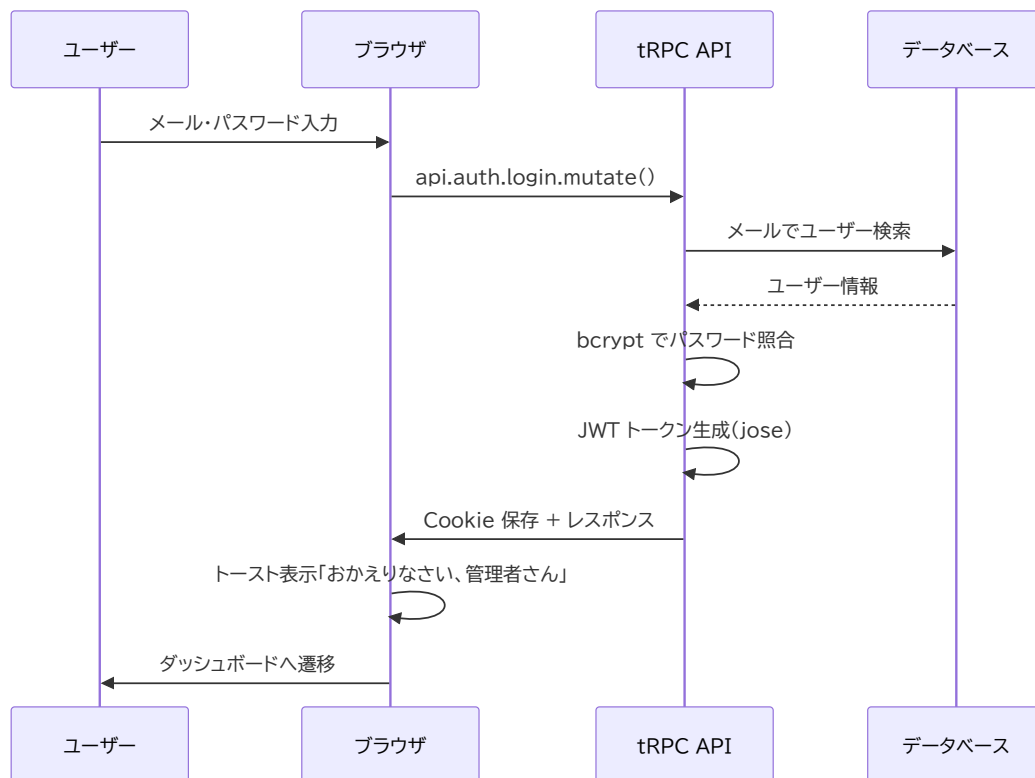


図 2: 認証フローの全体像

この図で追ってほしいのは、このログイン処理でパスワードを送るのが最初の1回だけという点です。ログイン後のリクエストでは、ブラウザは Cookie に入った JWT を送ります。パスワードも Cookie 内の JWT も、他人に渡してはいけない認証情報です。どちらかを盗まれると、本人になりすましてアクセスされるおそれがあります。ここからは、JWT をどの通信へ送るか、ブラウザ上の JavaScript からどう隠すかを確認します。

やること / やらないこと

やること	やらないこと
認証バックエンドを自分の手で作る	暗号化の数学的仕組みを理解する

やること	やらないこと
JWT・Cookie・middleware の仕組みを体験する	独自の暗号化実装を作る
DevTools で認証フローを目で確認する	データベース設計(scaffold 済み)

新しく学ぶ概念

概念	読み方	役割	例え
JWT	ジェイ・ダブリュー・ティー	ユーザー情報を署名付きで格納したトークン	遊園地のリストバンド。名前と有効期限入り
bcrypt	ビークリプト	パスワードを安全にハッシュ化	暗証番号を元に戻せないハッシュ値へ変換するマシン
HttpOnly Cookie	エイチティー・ティーピー・オンリー・クッキー	JS から読めない安全な Cookie	見えない場所に隠したリストバンド
tRPC	ティー・アール・ピー・シー	型安全な API フレームワーク	フロントとバックで同じメニュー表を共有する仕組み
ミドルウェア	middleware	リクエストの前処理(認証ガード等)	店の入口にいるガードマン

概念	読み方	役割	例え
rate limit	レート・リミット	短時間のログイン 失敗回数を制限 する	暗証番号を何度も間違え ると一時停止する仕組み

今日のゴールライン: JWT や bcrypt の数学的な仕組みまで理解する必要はありません。「ログインしたらトークンがもらえてそれで認証が通る」という流れを体験できたら OK。

実装ステップ一覧

ステップ	作業内容	読む時間の 目安	扱うファイル
Step 0	控えを取り、配布済みのDB列を確認し、共通部品を書き直す	8分	<code>password.ts</code> , <code>user-email-conflict.ts</code>
Step 1	<code>session.ts</code> を作り直す(JWT セッション管理)	12分	<code>src/lib/session.ts</code>

ステップ	作業内容	読む時間の 目安	扱うファイル
Step 2	trpc.ts を 作り直す (API の 土台)	10分	<code>src/server/api/trpc.ts</code>
Step 3	auth.ts を作り直す (認証ルー ター)	15分	<code>src/server/api/routers/auth.ts</code> + ヘルパー
Step 4	API を繋 ぎ直す (ルーター 登録 + HTTP ハ ンドラー)	5分	<code>src/server/api/root.ts</code> , <code>src/app/api/trpc/[trpc]/route.ts</code>
Step 5	middlew are.ts を 作る(ルー ト保護)	8分	<code>src/middleware.ts</code>
Step 6	ログインし て動作確認 する	5分	なし

ステップ	作業内容	読む時間の 目安	扱うファイル
	DevTool s で JWT		
Step 7	と Cookie を確認する	5分	なし

読む時間の合計(仮): 約 68 分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

Step 0: 控えを取り、配布済みの DB 列を確認し、共通部品を書き直す(読む目安: 8 分)

今日はすでに動いている 8 つのファイルの中身を書き直します。途中で貼り間違えても戻せるように先に控えを取ります。ターミナルで次を実行してください。

```
# filepath: ターミナル
mkdir -p ~/day07-backup
cp src/lib/session.ts src/server/api/trpc.ts \
  src/lib/password.ts \
  src/server/api/routers/auth.ts src/server/api/root.ts \
  src/server/api/routers/_helpers/select.ts \
  src/server/api/routers/_helpers/user-email-conflict.ts \
  "src/app/api/trpc/[trpc]/route.ts" \
  ~/day07-backup/
ls ~/day07-backup
```

`ls` で8つのファイル名が出れば控えが取れています。書き直しに失敗してやり直す場合だけ、次のコマンドで8ファイルを元に戻します。実行すると今日の編集内容を控えで上書きします。通常は実行せず、Step 1へ進んでください。

```
# filepath: ターミナル(復元が必要な場合だけ)
cp ~/day07-backup/session.ts src/lib/
cp ~/day07-backup/password.ts src/lib/
cp ~/day07-backup/trpc.ts src/server/api/
cp ~/day07-backup/auth.ts src/server/api/routers/
cp ~/day07-backup/root.ts src/server/api/
cp ~/day07-backup/select.ts src/server/api/routers/_helpers/
cp ~/day07-backup/user-email-conflict.ts src/server/api/routers/_helpers/
cp ~/day07-backup/route.ts "src/app/api/trpc/[trpc]/"
```

Step 5で新しく作った `src/middleware.ts` は、この控えには含まれません。上の操作は8ファイルの復元だけです。

Day 07: 認証バックエンドを作ってログインを動かそう

セッションの版数を判定するため、Day 01 の scaffold が作った

`prisma/schema.prisma` を開きます。`User` モデルに次の1行が1本だけあることを確認してください。すでにある行なので追加や `npm run db:push` は不要です。

```
// 読み比べ用: prisma/schema.prisma の User モデルにある1行
sessionVersion Int @default(0) @map("session_version")
```

`sessionVersion` の初期値を0にするのは、初めて作るユーザーにも版数を必ず持たせるためです。2本ある場合は追加分を消し、1本に戻します。

次に、scaffold が配布した `src/lib/password.ts` を開き、中身をすべて削除してから次の全文で置き換えます。追記すると `import` と `export` が二重になるので、必ず全文を置き換えてください。

```
// filepath: src/lib/password.ts
import { z } from 'zod';

export const BCRYPT_PASSWORD_MAX_BYTES = 72;

export const isPasswordWithinBcryptLimit = (password:
string): boolean =>
  new TextEncoder().encode(password).byteLength <=
BCRYPT_PASSWORD_MAX_BYTES;

export const createPasswordSchema = (
minimumLengthMessage: string) =>
  z
    .string()
    .min(8, minimumLengthMessage)
    .regex(/[A-Z]/,
'パスワードには大文字を含める必要があります')
    .regex(/[a-z]/,
'パスワードには小文字を含める必要があります')
    .regex(/[0-9]/,
'パスワードには数字を含める必要があります')
    .regex(/^[A-Za-z0-9]/,
'パスワードには特殊文字を含める必要があります')
    .refine(isPasswordWithinBcryptLimit,
'パスワードはUTF-8で72バイト以内にしてください');
```

Day 07: 認証バックエンドを作ってログインを動かそう

`TextEncoder` で文字列を UTF-8 のバイト列へ変換してから数えます。`bcrypt` が扱える上限は72文字ではなく72バイトです。ASCII 文字は通常1バイトですが、日本語や絵文字は1文字が複数バイトになります。そのため文字数だけでは上限を正しく判定できません。既存アカウントのログインは `loginSchema` の `min(1)` のままにし、72バイト条件を後から追加してもログインを妨げないようにします。

ほぼ同時に同じメールアドレスの登録が届いた場合に備えます。`scaffold` が配布した `src/server/api/routers/_helpers/user-email-conflict.ts` も中身をすべて削除し、次の全文で置き換えます。

```
// filepath: src/server/api/routers/_helpers/user-email-conflict.ts
import { Prisma } from '@prisma/client';

export const isUserEmailUniqueConstraintError = (error:
unknown): boolean => {
  if (!(error instanceof
Prisma.PrismaClientKnownRequestError) || error.code !==
'P2002') {
    return false;
  }

  const target = error.meta?.['target'];
  const modelName = error.meta?.['modelName'];
  return (
    Array.isArray(target) &&
    target.length === 1 &&
    target[0] === 'email' &&
    (modelName === undefined || modelName === 'User')
  );
};
```

P2002 だけでなく **User.email** の制約かまで確かめます。他の一意制約エラーをメール重複と誤って表示しないためです。

控えを8つ取るのは今日この8つすべてを中身ごと置き換えるためです。内訳は3種類あります。Step 1 から Step 4 で順に書き直すのが **session.ts** **trpc.ts** **auth.ts** **root.ts** の4つです。Step 3 と Step 4 で「中身を教材のコードへ置き換える」と指示するのが **select.ts** と **route.ts** の2つです。この Step

Day 07: 認証バックエンドを作ってログインを動かそう

で書き直すのが `password.ts` と `user-email-conflict.ts` の2つです。
`schema.prisma` は書き換えず、1行を確認するだけです。今日触るファイルはもう1つあります。Step 5 で作る `src/middleware.ts` です。これは今日はじめて作るファイルなので控えは要りません。

確認ポイント:

- `ls ~/day07-backup` に `session.ts` `password.ts` `trpc.ts` `auth.ts` `root.ts` `select.ts` `user-email-conflict.ts` `route.ts` の8つが出ます。
- `schema.prisma` の `User` モデルに `sessionVersion` が1本だけあります。

Step 1: `session.ts` を作り直す(JWT セッション管理・読む目安: 12分)

Step 1 から Step 4 のあいだ、アプリは動かない状態になります。開発サーバーを起動したまままだとまだ書き直していないファイル由来の英語のエラーが画面いっぱいに出ます。`npm run dev` を動かしているターミナルで `Ctrl+C` を押し、開発サーバーを止めます。Step 6 で起動し直します。

ゴール: ログイン状態を JWT トークンで管理する仕組みを作ります。

このファイルが認証の中心です。「トークンを作る」「トークンを読む」「Cookie に保存する」「Cookie から取り出す」——認証の基本操作が全部ここに入ります。

`src/lib/session.ts` を開き、中身をすべて削除してから作り直します。

1-1. インポートと秘密鍵の取得

```
// filepath: src/lib/session.ts
import { type JWPayload, jwtVerify, SignJWT } from
'jose';
import { cookies } from 'next/headers';
import { USER_ROLE, type UserRole } from
'./constant/roles';
import { env } from './env';

function getKey(): Uint8Array {
  const SECRET_KEY = env.JWT_SECRET;
  const encoded = new TextEncoder().encode(SECRET_KEY);
  return new Uint8Array(encoded);
}
```

コード	意味	例え
<code>jose</code>	JWT を扱うライブラリ	リストバンド製造機
<code>cookies()</code>	Next.js の Cookie 操作 API	ブラウザの Cookie 棚
<code>getKey()</code>	秘密鍵を Uint8Array に変換	店長の印鑑を取り出す

`TextEncoder` (文字列を UTF-8 バイト列へ変換するブラウザや Node.js 標準の道具)を使って、秘密鍵の文字列をバイナリのバイト列へ変換します。`getKey()` の返すこの鍵が今日作る認証の土台です。署名に使う鍵と検証に使う鍵が同じなので鍵を知っている人は誰でも正規のトークンを作れます。つまり `JWT_SECRET` が漏れた

Day 07: 認証バックエンドを作ってログインを動かそう

時点で、攻撃者は好きな `userId` と `role` を書いたトークンを自作でき、パスワードは要らなくなります。`.env` をリポジトリへコミットしないのはこの 1 行を守るためです。

`JWT_SECRET` は `.env` に scaffold が設定済み。32 文字以上の文字列で、本番では必ず変更します。

1-2. 型定義と定数

```
// filepath: src/lib/session.ts(続き)
export interface SessionPayload {
  userId: string;
  email: string;
  role: UserRole;
  version: number;
  exp: number;
}

export interface SessionUser {
  id: string;
  email: string;
  role: UserRole;
  version: number;
}

const POSTGRES_INTEGER_MAX = 2_147_483_647;
const COOKIE_NAME = 'session';
const COOKIE_MAX_AGE = 60 * 60 * 24 * 7; // 7日間
```

`version` は `sessionVersion` (セッション版数: ログインごとに付与され、パスワード変更やアカウント無効化時に加算される整数値) です。トークン発行時にDBの現在値を埋め込み、後の認証チェックでDBの値と照合します。また、

`POSTGRES_INTEGER_MAX` は PostgreSQL の 4 バイト整数型の最大値 (`2,147,483,647`) です。異常な値や巨大な数値がトークンに紛れ込まないように上限として保持します。

型を 2 つに分けているのはトークンの都合を画面へ持ち込まないためです。

`SessionPayload` には `exp` (トークンの有効期限を表す数値) のような、トークンの管理にしか使わない値が入っています。画面へ同じ型をそのまま渡すと期限の持ち方を変えただけで画面側のコードまで直すことになります。そこで `SessionUser` を別に用意して画面が実際に表示する値だけを渡します。

1-3. 型ガードと署名

```
// filepath: src/lib/session.ts(続き)
function readSessionPayload(payload: JWTPayload):
SessionPayload | null {
  if (
    typeof payload['userId'] !== 'string' ||
    typeof payload['email'] !== 'string' ||
    (payload['role'] !== USER_ROLE.USER &&
payload['role'] !== USER_ROLE.ADMIN) ||
    typeof payload['exp'] !== 'number'
  ) return null;

  const rawVersion = Object.hasOwn(payload, 'version') ?
payload['version'] : 0;
  const isVersionValid =
```

```
typeof rawVersion === 'number' &&  
Number.isInteger(rawVersion) &&  
rawVersion >= 0 &&  
rawVersion <= POSTGRES_INTEGER_MAX;  
if (!isVersionValid) return null;  
  
return {  
  userId: payload['userId'],  
  email: payload['email'],  
  role: payload['role'],  
  version: rawVersion,  
  exp: payload['exp'],  
};  
}
```

`readSessionPayload` は JWT の中身(クレーム)を検証して安全に取り出します。新規発行トークンには `version` が必須です。バージョン管理導入前の古い JWT から届いた場合だけ、`Object.hasOwn` でプロパティ欠落を検知して `0` へフォールバックします。一方、`version` が文字列や小数、負の数、上限超過などの異常なクレームであれば `null` を返して拒否します。中身を確認めているのは JWT が「改ざんされていない」ことしか保証しないためです。署名が正しくても古い版のアプリが項目を欠いて発行したトークンなら欠けたまま届きます。ここで確かめずに先へ進むと不正な値が権限判定へ流れます。

確認ポイント:

- ☐ `readSessionPayload` が `userId` / `email` / `role` / `exp` と `version` を確認しています。

```
// filepath: src/lib/session.ts(続き)
export async function signSessionToken(
  payload: SessionPayload,
): Promise<string> {
  const jwtPayload: Record<string, unknown> = {
    userId: payload.userId,
    email: payload.email,
    role: payload.role,
    version: payload.version,
    exp: payload.exp,
  };

  return await new SignJWT(jwtPayload)
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('7d')
    .sign(getKey());
}
```

コード	意味	例え
<code>readSessionPayload()</code>	JWT の中身が正しい形式か検証	書類の記入漏れ・異常値チェック
<code>SignJWT</code>	署名付き JWT を作成	リストバンドに情報を刻印する機械
<code>alg: 'HS256'</code>	署名アルゴリズム	偽造防止の特殊インクの種類

Day 07: 認証バックエンドを作ってログインを動かそう

コード	意味	例え
<code>setExpirationTime('7d')</code>	7 日間有効	リストバンドの有効期限 シール
<code>sign(getKey())</code>	秘密鍵で署名	店長のハンコで正式認定

`jwtPayload` に入れているのは `userId` / `email` / `role` / `version` / `exp` の 5 つだけです。この教材の JWT は署名付きで、暗号化していません。受け取った人は中身を読み取れます。パスワードや電話番号を足したくなくても入れてはいけないのはそのためです。最後の `.sign(getKey())` が付ける署名は中身を隠すものではなく、書き換えられていないことを示す封印だと考えてください。

今日出てくる 3 つの言葉は似ていますが役割が違います。

処理	何をするか	元に戻せるか
署名	内容が書き換えられていないことを示す印を付ける	内容は最初から見えている
ハッシュ化	パスワードを元に戻せない値へ変換する	戻せない(照合だけできる)
暗号化	内容を読めない形に変換する	鍵があれば戻せる

このアプリでしているのは署名とハッシュ化だけです。JWT の中身は誰でも読めます。

1-4. 署名の検証(トークンを読む)

```
// filepath: src/lib/session.ts(続き)
export async function verifySessionToken(
  token: string,
): Promise<SessionPayload | null> {
  try {
    const { payload } = await jwtVerify(token, getKey(), {
      algorithms: ['HS256'],
    });

    const session = readSessionPayload(payload);
    if (!session) {
      console.error('Invalid session payload structure');
      return null;
    }

    return session;
  } catch {
    console.error('Failed to verify session token');
    return null;
  }
}
```

`signSessionToken` がトークンへ署名して作り、`verifySessionToken` がその署名を確かめてから中身を読みます。この2つは対になる操作です。読む側の中心は `jwtVerify` で、秘密鍵と合う署名が付いているかを確認めます。ここを飛ばして中身だけ取り出す作りになると、攻撃者が他人の `userId` を書いたトークンで、その人になりすましてしまいます。後でDBから権限を取り直しても、本人かどうかの確

Day 07: 認証バックエンドを作ってログインを動かそう

認にはなりません。署名が通っても項目が足りない場合や異常値は

`readSessionPayload` で弾き、期限切れなどで例外が出たときは `catch` が `null` を返します。ログイン済みとみなしてよいのはこの関数が `null` 以外を返したときだけです。

1-5. セッション操作(作成・取得・削除・検証)

```
// filepath: src/lib/session.ts(続き)
export async function saveSessionCookie(
  token: string,
): Promise<void> {
  const cookieStore = await cookies();
  cookieStore.set(COOKIE_NAME, token, {
    httpOnly: true,
    secure:
      process.env['NODE_ENV'] === 'production'
      && process.env['PLAYWRIGHT_TEST'] !== '1',
    sameSite: 'strict',
    maxAge: COOKIE_MAX_AGE,
    path: '/',
  });
}
```

トークンを Cookie へ書き込む処理だけを、別の関数へ切り出しています。ログイン直後と、あとから発行し直すときの両方から呼ぶので書き込みの条件を 1 か所にそろえておくためです。ここでは5つの設定で、Cookieの送信範囲・読み取りの制限・

保存期間を決めます。たとえば `httpOnly: true` を外すとページに紛れ込んだスクリプトが `document.cookie` からトークンを読み出せてしまいます。設定それぞれの意味はこの節の最後の表へまとめてあります。

確認ポイント:

- ☐ Cookie 設定を `saveSessionCookie` に分けて書けています。

```
// filepath: src/lib/session.ts(続き)
export async function createSession(
  user: SessionUser,
): Promise<string> {
  const expiresAt =
    Math.floor(Date.now() / 1000) + COOKIE_MAX_AGE;
  const payload: SessionPayload = {
    userId: user.id,
    email: user.email,
    role: user.role,
    version: user.version,
    exp: expiresAt,
  };

  const token = await signSessionToken(payload);
  await saveSessionCookie(token);

  return token;
}
```

Day 07: 認証バックエンドを作ってログインを動かそう

`exp` を秒で数えているのは JWT の決まりが秒を使うためです。`Date.now()` はミリ秒を返すので 1000 で割ります。`signSessionToken` は最後に `.setExpirationTime('7d')` を呼ぶため、payload に入れた `exp` はその期限で上書きされます。JWT の検証に使われる期限は、署名するときに設定した 7 日後です。

期限をトークンの中と Cookie の両方に持たせているのは片方だけでは足りないからです。Cookie の期限だけだと利用者が手元で Cookie の期限を書き換えて延命できます。トークンの中にも `exp` を入れておけば署名で守られているので書き換えられません。

確認ポイント:

- `createSession` から `saveSessionCookie(token)` を呼んでいます。

```
// filepath: src/lib/session.ts(続き)
export async function getSession():
Promise<SessionPayload | null> {
  const cookieStore = await cookies();
  const token = cookieStore.get(COOKIE_NAME)?.value;

  if (!token) {
    return null;
  }

  return await verifySessionToken(token);
}
```

`cookieStore.get(COOKIE_NAME)?.value` は Cookie が見つからないと `undefined` になります。 `verifySessionToken` の引数は文字列なので、この値はそのまま渡せません。ここではログインしていない状態を `null` で表し、JWT の検証へ進む前に返します。Cookie が無いリクエストで、検証失敗のログを出さずに済みます。

確認ポイント:

- ☐ Cookie がない場合に `null` を返しています。

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/lib/session.ts(続き)
export async function deleteSession(): Promise<void> {
  const cookieStore = await cookies();
  cookieStore.delete(COOKIE_NAME);
}

export async function verifySession():
Promise<SessionUser | null> {
  const session = await getSession();

  if (!session) {
    return null;
  }

  return {
    id: session.userId,
    email: session.email,
    role: session.role,
    version: session.version,
  };
}
```

PLAYWRIGHT_TEST は、自動テストをHTTPで動かすための合図です。通常の学習では設定しません。本番で **1** を設定すると **secure** が無効になるので、本番の環境変数にも追加しないでください。

Cookie 設定の意味:

設定	値	なぜ必要か
<code>httpOnly</code>	<code>true</code>	JavaScript から直接読めなくし、XSS 発生時にセッション Cookie を盗まれる経路を狭める
<code>secure</code>	本番のみ <code>true</code>	HTTPS でのみ送信して盗聴を防ぐ
<code>sameSite</code>	<code>'strict'</code>	別サイトからのリクエストに Cookie を付けない
<code>maxAge</code>	7 日間	Cookie を保存する期間
<code>path</code>	<code>/</code>	このサイトのすべてのパスで Cookie を送る

確認ポイント:

- ☐ `src/lib/session.ts` を教材のコードで作り直しました。
- ☐ トークンと Cookie を扱う `signSessionToken` / `verifySessionToken` / `saveSessionCookie` があります。
- ☐ セッションを扱う `createSession` / `getSession` / `deleteSession` / `verifySession` があります(合わせて7関数)。
- ☐ この時点ではまだ `npm run dev` しなくて OK

学んだこと: JWT は「誰が」「いつまで」「どの権限で」ログインしているかを、署名付きで保持する仕組みです。

Day 07: 認証バックエンドを作ってログインを動かそう

Step 2: trpc.ts を作り直す(API の土台・読む目安: 10 分)

ゴール: tRPC の初期設定と、public / protected / admin の 3 種類の API を定義します。

Day 05 のログイン画面は `api.auth.login.useMutation()` でサーバーを呼んでいました。その呼び出しを受け止めるサーバー側の土台がこれから作り直すこのファイルです。

`src/server/api/trpc.ts` を開き、中身をすべて削除してから作り直します。

2-1. コンテキスト作成

```
// filepath: src/server/api/trpc.ts
import { initTRPC, TRPCError } from '@trpc/server';
import superjson from 'superjson';
import { ZodError } from 'zod';
import { USER_ROLE } from '@/lib/constant/roles';
import { prisma } from '@/lib/prisma';
import { getSession } from '@/lib/session';

export const createTRPCContext = async (
  opts: { headers: Headers },
) => {
  const session = await getSession();

  return {
    session,
    ...opts,
  };
};

export type Context = Awaited<
  ReturnType<typeof createTRPCContext>
>;
```

コンテキストとは1回のリクエストのあいだ、どのAPI処理からも参照できる共通の入れ物です。ログイン中のユーザーをここへ入れておくのはAPIを1つ増やすたびにCookieの読み取りとJWTの検証を書き足さずに済ませるためです。判定は1リクエストにつき1回だけ走り、どのAPI処理も同じ結果を受け取ります。

2-2. tRPC インスタンス初期化

```
// filepath: src/server/api/trpc.ts(続き)
const t = initTRPC.context<Context>().create({
  transformer: superjson,
  errorFormatter({ shape, error }) {
    return {
      ...shape,
      data: {
        ...shape.data,
        zodError:
          error.cause instanceof ZodError
            ? error.cause.flatten()
            : null,
      },
    };
  },
});
```

コード	意味
<code>superjson</code>	Date や BigInt を JSON で送れるようにする変換器
<code>errorFormatter</code>	zod のバリデーションエラーを使いやすい形に整形

2-3. 認証ミドルウェアとプロシージャ

```
// filepath: src/server/api/trpc.ts(続き)
const isAuthenticated = t.middleware(
  async ({ ctx, next }) => {
    if (!ctx.session?.userId) {
      throw new TRPCError({
        code: 'UNAUTHORIZED',
        message: 'ログインが必要です',
      });
    }

    const currentUser = await prisma.user.findUnique({
      where: { id: ctx.session.userId },
      select: {
        id: true,
        email: true,
        role: true,
        isActive: true,
        sessionVersion: true,
      },
    });
```

前半は `ctx.session?.userId` が無ければ `UNAUTHORIZED` を投げて先へ進ませない門番です。目を留めてほしいのはその直後に `prisma.user.findUnique` でデータベースを引き直しているところです。JWT の有効期限は 7 日間あるので、たった今アカウントを止めたりパスワードを変更したりしても相手の手元にあるトークン

Day 07: 認証バックエンドを作ってログインを動かそう

ンは形の上では有効なままです。トークンの中身だけを信じる作りだと無効化した相手を最大 7 日間止められません。リクエストのたびにデータベースを見に行くのはこの時間差を埋めるためです。

確認ポイント:

- ☐ セッションの `userId` で DB のユーザーを取り直しています。

```
// filepath: src/server/api/trpc.ts(続き)
if (!currentUser) {
  throw new TRPCError({
    code: 'UNAUTHORIZED',
    message: 'ユーザーが見つかりません',
  });
}

if (!currentUser.isActive) {
  throw new TRPCError({
    code: 'FORBIDDEN',
    message: 'このアカウントは無効化されています',
  });
}

if (currentUser.sessionVersion !==
ctx.session.version) {
  throw new TRPCError({
    code: 'UNAUTHORIZED',
    message:
      'セッションが無効になりました。再度ログインしてください',
  });
}
```

分岐を分けているのは返すべき答えと理由が違うからです。ユーザーが見つからない側は退会などでレコードが消えた状態にあたります。相手が誰なのか確かめられないので **UNAUTHORIZED** を返し、ログインし直せば通る可能性を残します。

isActive が **false** の側は誰かは分かっているが利用を止めてある状態なので

Day 07: 認証バックエンドを作ってログインを動かそう

FORBIDDEN を返します。 **sessionVersion** が一致しない側は、別端末でのパスワード変更や管理者による操作でトークンが無効化された状態です。失効を知らせるために **UNAUTHORIZED** を返します。

今日は配布スターターの DB にセッション版数の列があるため、Step 0 で確認したとおり列の追加は不要です。将来、この列が無い既存の本番 DB へ機能を追加する場合は、マイグレーション(列の追加)を先に適用し、そのあとに対応コードをデプロイします。また、セッション失効の保証境界にも注意が必要です。失効のコミットが完了したあとに DB 認証チェックを開始したリクエストは確実に弾かれますが、すでに認証チェックを通過して実行中のリクエストを取り消すものではありません。

確認ポイント:

- ☐ DB にユーザーがない場合、無効化済み、セッション版数不一致を分けています。

```
// filepath: src/server/api/trpc.ts(続き)
return next({
  ctx: {
    session: {
      ...ctx.session,
      email: currentUser.email,
      role: currentUser.role,
    },
  },
});
```

`email: currentUser.email` と `role: currentUser.role` で上書きしているのがこの節で重要な行です。トークンに入っている値はログインした瞬間の値で固定されています。あとから管理者権限を外したりメールアドレスを変えたりしても相手のトークンの中身は自動で書き換わりません。毎回データベースの最新値を取り直して差し替えるので権限変更が即座に反映されます。

確認ポイント:

- ☐ 未ログイン時に `UNAUTHORIZED` を返す分岐があります。
- ☐ DB から `isActive` と `sessionVersion` を確認しています。
- ☐ DB の最新 `email` と `role` でコンテキストを上書きしています。

```
// filepath: src/server/api/trpc.ts(続き)
const isAdmin = t.middleware(async ({ ctx, next }) => {
  if (ctx.session?.role !== USER_ROLE.ADMIN) {
    throw new TRPCError({
      code: 'FORBIDDEN',
      message: '管理者権限が必要です',
    });
  }

  return next({ ctx });
});
```

`isAdmin` はこれ単体では守りになりません。見ているのは

`ctx.session?.role` だけです。この値が最新の権限を反映しているのは、1つ前の `isAuthenticated` がデータベースの内容で上書きしているからです。

`isAuthenticated` を通さずにこれだけを使うと権限を下げた直後のユーザーが

Day 07: 認証バックエンドを作ってログインを動かそう

古いトークンに残った `ADMIN` のまま管理者向け API を呼べてしまいます。本当の守りは `isAuthenticated` 側にあり、`isAdmin` はそこで確定した値をふるいにかける役です。次のコードで `.use()` を並べる順番がそのまま守りの順番になります。

確認ポイント:

- ☐ 管理者以外を `FORBIDDEN` にする `isAdmin` があります。

```
// filepath: src/server/api/trpc.ts(続き)
export const createTRPCRouter = t.router;
export const publicProcedure = t.procedure;
export const protectedProcedure =
  t.procedure.use(isAuthenticated);
export const adminProcedure =
  t.procedure.use(isAuthenticated).use(isAdmin);
export const createCallerFactory = t.createCallerFactory;
```

3 種類の API:

種別	認証	使う場面	API 例
<code>publicProcedure</code>	不要	誰でも呼べる	ログイン、登録
<code>protectedProcedure</code>	必須	ログインユーザーのみ	タスク操作、プロジェクト管理
<code>adminProcedure</code>	管理者のみ	管理機能	ユーザー管理

`isAuthenticated` ミドルウェアはCookie のセッション情報だけでなく DB からユーザーの最新状態を取得します。アカウントが無効化されていたらここで弾きます。

確認ポイント:

- ☐ `src/server/api/trpc.ts` を教材のコードで作り直しました。
- ☐ `publicProcedure` / `protectedProcedure` / `adminProcedure` の 3 つが export されています。

学んだこと: tRPC のミドルウェアで「ログイン必須」「管理者のみ」といった認証制御を API 定義にチェーン(`.use()`)するだけで追加できます。

Step 3: auth.ts を作り直す(認証ルーター・読む目安: 15 分)

ゴール: ログイン・登録・ログアウト・セッション取得・現在のユーザー取得の 5 つの API を作ります。

ここが今日のメインです。配布スターターには Day 05 と Day 06 の画面を先に動かすため完成済みの認証 API が入っています。ここでは動いているコードを答えとして眺めるだけにせず、`auth.ts` の中身を削除し、手順どおりに自分で作り直します。

まず共通のヘルパーを作ります。

3-0. select ヘルパーを作る

`src/server/api/routers/_helpers/select.ts` を開き、教材のコードと見比べます。内容が異なる場合は中身を教材のコードへ置き換えてください。

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/routers/_helpers/select.ts
import { z } from 'zod';
import { PROJECT_MEMBER_ROLE } from
'@/lib/constant/roles';

export const USER_SELECT = {
  id: true,
  name: true,
  email: true,
  avatar: true,
} as const;

export const USER_DETAIL_SELECT = {
  id: true,
  email: true,
  name: true,
  avatar: true,
  role: true,
  isActive: true,
} as const;

export const projectMemberRoleSchema =
  z.nativeEnum(PROJECT_MEMBER_ROLE);
```

Prisma(読み方はプリズマ、TypeScript からデータベースを操作するための道具)の `select` は取り出す項目をこちらで指定する書き方です。これを API ごとに書くと同じユーザー情報でも API によって返る項目がばらつきます。1 か所でも書き

Day 07: 認証バックエンドを作ってログインを動かそう

間違えればその API だけが外へ出したくない項目まで返します。定数にまとめておけば項目を足すときに直す場所は 1 つで済みます。 `as const` を付けると中身が読み取り専用になり、返り値の型もここで並べた項目どおりに決まります。

3-1. auth.ts のインポートとバリデーション

`src/server/api/routers/auth.ts` を開き、中身をすべて削除してから作り直します。ファイル自体は削除せず、空になった同じファイルへ次のコードを書いてください。

```
// filepath: src/server/api/routers/auth.ts
import { TRPCError } from '@trpc/server';
import bcrypt from 'bcryptjs';
import { z } from 'zod';
import { USER_ROLE } from '@lib/constant/roles';
import { createPasswordSchema } from '@lib/password';
import { prisma } from '@lib/prisma';
import {
  checkLoginRateLimit,
  extractClientIp,
  rateLimitToTRPCError,
  recordLoginSuccess,
} from '@lib/rate-limit';
import { createSession, deleteSession, type SessionUser }
from '@lib/session';
import { createTRPCRouter, protectedProcedure,
publicProcedure } from '../trpc';
import { USER_DETAIL_SELECT } from '../_helpers/select';
import { isUserEmailUniqueConstraintError } from
'../_helpers/user-email-conflict';
```

取り込んだ道具は役割ごとに分かれています。 `bcrypt` はパスワードの照合、 `z` は入力の検証、 `prisma` はデータベースへの問い合わせに使います。

`@/lib/rate-limit` から取り込んだ 4 つはログイン失敗の回数を数えて当てずっぽうの試行を止めるための道具です。最後の `publicProcedure` はログイン前でも呼べる手続き、 `protectedProcedure` はログイン済みだけが呼べる手続きを作ります。ログインと登録に `publicProcedure` を使わないとまだログインしていない人がログインできない API になってしまいます。

確認ポイント:

- ☐ 認証 API に必要な import が揃っています。

```
// filepath: src/server/api/routers/auth.ts(続き)
const loginSchema = z.object({
  email:
z.string().email('有効なメールアドレスを入力してください'),
  password: z.string().min(1,
'パスワードを入力してください'),
});
```

このあと出てくる `registerSchema` と見比べると `password` の条件が `min(1)` だけになっています。ログイン側で 8 文字以上や記号必須まで課すといま登録されているパスワードでログインできなくなる人が出ます。登録時の条件はあとから厳しくすることがあります。付属の初期データに入っている `password123` も、いまの `registerSchema` の条件は満たしません。ここでの検証は空の値をデータベース照合まで流さないための入口チェックにとどめます。正しいかどうかを決めるのはこのあとの `bcrypt.compare` の仕事です。

Day 07: 認証バックエンドを作ってログインを動かそう

確認ポイント:

- ☐ ログイン入力には email と password を検証しています。

```
// filepath: src/server/api/routers/auth.ts(続き)
const registerSchema = z.object({
  name: z.string().min(1, '名前を入力してください'),
  email:
    z.string().email('有効なメールアドレスを入力してください'),
  password:
    createPasswordSchema('パスワードは8文字以上で入力してください'),
});
```

zod でバリデーションを定義しておくと tRPC が自動で入力チェックしてくれます。フロント側でもバックエンド側でも同じスキーマを使えます。

3-2. エラーハンドラーとログイン処理

```
// filepath: src/server/api/routers/auth.ts(続き)

function handleUnexpectedError(context: string, error:
unknown): never {
  console.error('[auth] unexpected error', { context,
error });

  throw new TRPCError({
    code: 'INTERNAL_SERVER_ERROR',
    message: ` ${context}中にエラーが発生しました。時間をおいてお試しください。`,
    cause: error,
  });
}
```

戻り値の型が `never`なのはこの関数が値を返さず必ず例外を投げるからです。呼んだ側はこの行から先へ進まないという型の上でも分かります。本物のエラーは `console.error`でサーバーのログにだけ残し、画面へ返すのは決まった文言だけにします。データベースのエラー文をそのまま返すとテーブル名や列名といった内部の作りが相手に見えてしまうためです。末尾の `cause: error`は原因を捨てずに開発時の調査へ残しておくための指定です。

確認ポイント:

- ☐ 予期しないエラーを `TRPCError`に変換しています。

```
// filepath: src/server/api/routers/auth.ts(続き)
export const authRouter = createTRPCRouter({
  login: publicProcedure.input(loginSchema).mutation(
    async ({ input, ctx }) => {
      const ip = extractClientIp(ctx.headers);

      // brute force 対策: 直近 15 分の失敗回数を
      // email 単独 / email×IP / IP 単独の3軸で rate-limit。

      // checkLoginRateLimit は許可と同時に失敗行を先取りで記録する
      // (成功時は recordLoginSuccess が削除)
      const limitResult = await checkLoginRateLimit(
        input.email, ip);
      if (!limitResult.allowed) {
        throw rateLimitToTRPCError(limitResult);
      }
    }
  )
});
```

ログイン処理の最初の仕事はデータベースを引くことではなく回数を数えることです。 `checkLoginRateLimit` を `prisma.user.findUnique` より前に置くのは当てずっぽうのパスワードを何万回も試す攻撃を、照合へ届く前に止めるためです。数える軸は `extractClientIp` で取り出した接続元 IP と、送られてきたメールアドレスの両方です。IP だけで数えると接続元を変えながら同じアカウントを狙う手口を数え落とします。逆にメールだけで数えると1つのIPから大量のアカウントを試す手口が素通りします。

確認ポイント:

- ☐ IP を取り出し、ログイン試行回数を先に確認しています。

```
// filepath: src/server/api/routers/auth.ts(続き)
try {
  const user = await prisma.user.findUnique({
    where: { email: input.email },
  });

  if (!user?.password) {

// 失敗は checkLoginRateLimit が先取りで記録済みのため、
// ここでは追加記録しない
    throw new TRPCError({
      code: 'UNAUTHORIZED',
      message:
        'メールアドレスまたはパスワードが正しくありません',
    });
  }

  const isValid = await bcrypt.compare(
    input.password, user.password);
```

`user?.password` という 1 つの条件で、ユーザーが存在しない場合とパスワードが登録されていない場合をまとめて弾いています。返す文言はこのあとのパスワード不一致とまったく同じです。片方だけ「そのメールアドレスは登録されていません」と返すと攻撃者は総当たりで有効なメールアドレスの一覧を作れてしまいます。照合に `===` ではなく `bcrypt.compare` を使うのはデータベースに入っているのがハッシュ化された文字列で、入力された生のパスワードとは一致しないためです。

`bcrypt.compare` は入力を同じ手順で変換してから見比べます。

Day 07: 認証バックエンドを作ってログインを動かそう

確認ポイント:

- ☐ ユーザー有無とパスワード照合を同じエラーで確認しています。

```
// filepath: src/server/api/routers/auth.ts(続き)

    if (!isPasswordValid) {

// 失敗は checkLoginRateLimit が先取りで記録済みのため、
// ここでは追加記録しない
        throw new TRPCError({
            code: 'UNAUTHORIZED',
            message:
'メールアドレスまたはパスワードが正しくありません',
        });
    }

    if (!user.isActive) {
        // 無効判定はパスワード照合が通ったあとに行う。
        // 先に判定すると、正しいパスワードを知らなくても
        // 「無効化されたアカウントが存在する」ことを
        // 列挙できてしまうため。
        throw new TRPCError({
            code: 'FORBIDDEN',
            message: 'このアカウントは無効化されています',
        });
    }
}
```


`isActive` の判定をパスワード照合より後ろへ置いた理由はコード中のコメントのとおりです。先に判定するとパスワードを知らない相手でも「このメールアドレスは無効化済みのアカウントとして実在する」と読み取れてしまいます。判定を 1 つ入れ替えるだけで、隠しておきたい事実が漏れます。認証の処理では書いてある順番そのものが守りの一部になります。上から読んだときに通過条件が厳しい順に並んでいるかを確かめてください。

確認ポイント:

- ☐ パスワード照合後に `isActive` を確認しています。

```
// filepath: src/server/api/routers/auth.ts(続き)

const sessionUser: SessionUser = {
  id: user.id,
  email: user.email,
  role: user.role,
  version: user.sessionVersion,
};

// セッション発行の前に成功記録を確定させる。
// 順序が逆だと、記録の失敗で 500 を返す一方で
// 認証済みセッションだけが残るため。
await recordLoginSuccess(input.email, ip);
await createSession(sessionUser);
```

ここまで来た人だけがログイン成功として扱われます。 `recordLoginSuccess` を `createSession` より先に呼ぶのは Cookie を配ったあとで失敗記録の削除に失敗するとログインできた本人が失敗回数に縛られたまま残るからです。記録を先に確

Day 07: 認証バックエンドを作ってログインを動かそう

定させておけば途中で落ちても発行済みのセッションだけが宙に浮く事態を避けられます。`SessionUser` へ詰めるのは `id` / `email` / `role` / `version` です。DB から読み出した現在の `sessionVersion` を渡すことで、トークンに現在のセッション版数を埋め込みます。ここへ `password` を足すとハッシュ化済みとはいえトークンの中身として誰でも読める場所へ出てしまいます。

確認ポイント:

- ☐ `sessionVersion` を含む `SessionUser` を作成しています。
- ☐ 成功記録を確定してからセッションを発行しています。

```
// filepath: src/server/api/routers/auth.ts(続き)

return {
  user: {
    id: user.id,
    email: user.email,
    name: user.name,
    avatar: user.avatar,
    role: user.role,
  },
};
} catch (error) {
  if (error instanceof TRPCError) throw error;
  handleUnexpectedError('ログイン処理', error);
}
}),
```

返しているのは画面表示に要る 5 項目だけで、`user` をそのまま返してはいけません。 `prisma.user.findUnique` の戻り値にはハッシュ化済みのパスワードも入っており、返してしまうと総当たりの材料を相手へ渡すことになります。 `catch` の中で `TRPCError` だけを投げ直しているのは `UNAUTHORIZED` のような意図した失敗を 500 エラーへ塗り替えないためです。それ以外の想定外だけが `handleUnexpectedError` へ回り、ログには残しつつ当たり障りのない文言に置き換わります。

ログイン処理の流れ:

Day 07: 認証バックエンドを作ってログインを動かそう

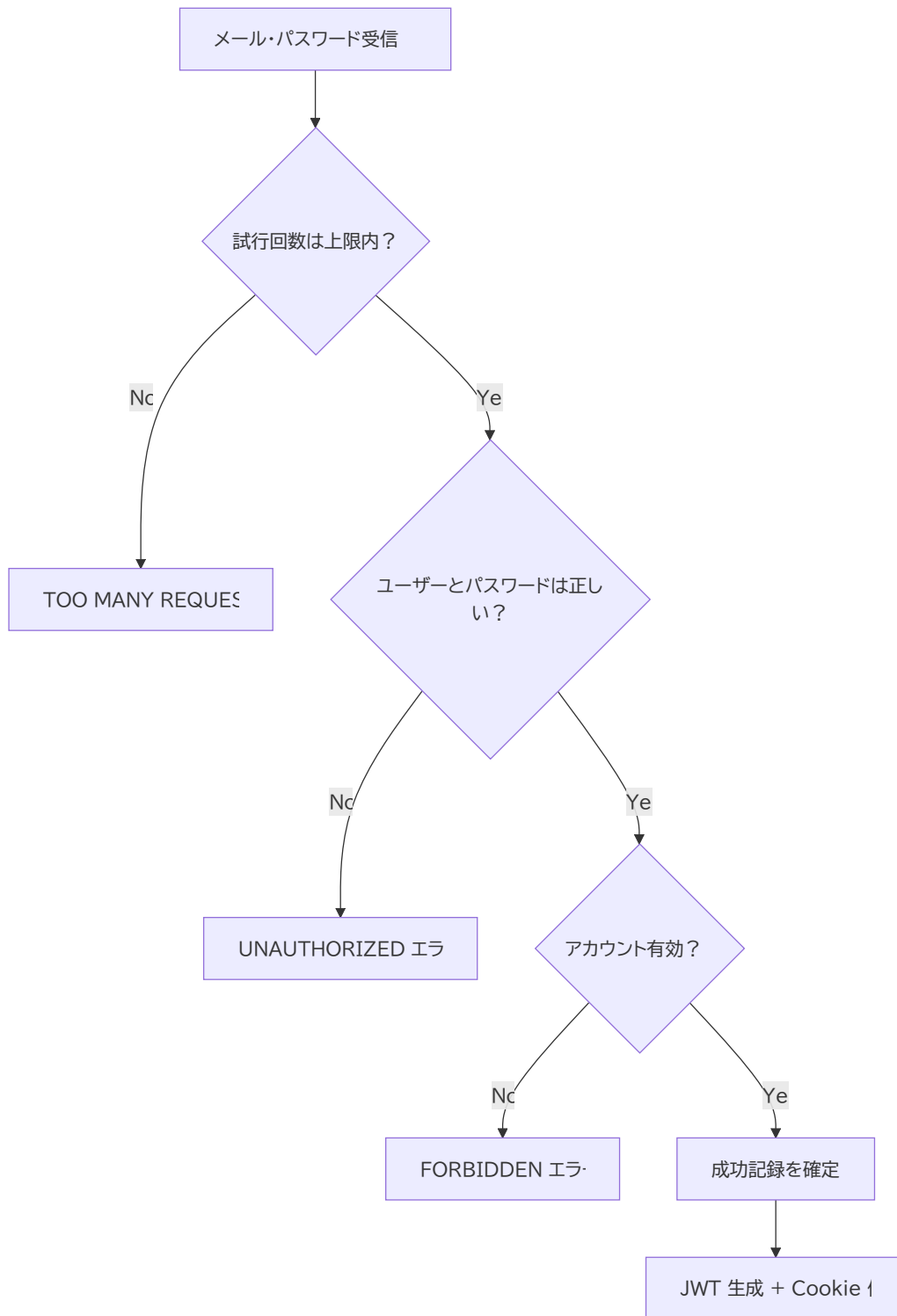


図3: 3-2. エラーハンドラーとログイン処理



The image shows a login form titled "ログイン" (Login) with the subtitle "アカウントにログインしてください" (Please login to your account). At the top is a blue circular icon with a white padlock. Below the title, there is a red-bordered box containing an error message: "① エラー" (Error) and "メールアドレスまたはパスワードが正しくありません" (Email address or password is incorrect). The error message is preceded by a red circle with the number "1". Below the error box, there are two input fields: "メールアドレス" (Email address) with the value "admin@example.com" and "パスワード" (Password) with masked characters ".....". A blue "ログイン" (Login) button is positioned below the password field. At the bottom, there is a link: "アカウントをお持ちでない方は [こちら](#)" (If you don't have an account, click [here](#)).

図 4: ログイン画面の上に「エラー メールアドレスまたはパスワードが正しくありません」という赤いメッセージが出ている状態

赤い枠は、いま書いた `login` が返した文言です。Day 05 では配布スターターの認証処理を使いました。今日は自分で書き直した処理から同じ文言を返せるか確認します。

なぜ同じエラーメッセージ? 「メールが存在しない」と「パスワードが違う」を区別すると攻撃者に「このメールは登録済み」と教えてしまいます。セキュリティのために同じメッセージを返します。

`checkLoginRateLimit` は同じメールや IP から短時間に失敗が続いたときパスワード照合へ進む前に止めます。成功時は `recordLoginSuccess` で失敗記録を削除してからセッションを発行します。順序を逆にすると記録の削除に失敗したのに認証 Cookie だけが残るためこの順にします。

ここで塞がっていないこと: ログインは「メールが無い」と「パスワードが違う」を同じ文言にしていますがこのあと書く `register` は重複したメールに対して「このメールアドレスは既に登録されています」を返します。つまり登録の側から「そのメールが使われているか」は分かってしまいます。しかも `register` は `checkLoginRateLimit` を通しません。実務では登録でも同じ回数制限をかけるか、重複かどうかを返さずメールで案内する作りにします。この教材では初心者が登録に失敗した理由を分かるようにするためあえて明示する形を採っています。

3-3. 登録・ログアウト・セッション取得

ここから先の「(続き)」のブロックは `auth.ts` の末尾に続けて貼ります。いまの `auth.ts` は Step 3-2 で書いた `}},` で終わっていて `createTRPCRouter({` はまだ閉じていません。閉じ括弧の `});` は Step 3-3 の最後のブロックで書くのでそれまでは自分で足さないでください。

```
// filepath: src/server/api/routers/auth.ts(続き)
register: publicProcedure
  .input(registerSchema)
  .mutation(async ({ input }) => {
    try {
      const existing = await prisma.user.findUnique({
        where: { email: input.email },
      });
```

登録処理の最初は同じメールアドレスがすでに使われていないかの確認です。ただしこの検索は重複を止める最後の砦ではありません。ほぼ同時に 2 件の登録が届くとどちらも「まだ無い」と判断して通り抜ける瞬間があるからです。最終的に重複を止めるのは `schema.prisma` で `email` に付けてある一意制約のほうです。ここでの検索は利用者へ分かりやすいエラーを返すための確認だと考えてください。

同時登録で一意制約に拒否された側は、先ほど作った

`isUserEmailUniqueConstraintError` で `User.email` の重複かを確認めます。その場合は事前確認と同じ `CONFLICT` を返します。

確認ポイント:

- ☐ 登録前に同じメールアドレスのユーザーを探しています。

```
// filepath: src/server/api/routers/auth.ts(続き)
    if (existing) {
        throw new TRPCError({
            code: 'CONFLICT',
            message:
                'このメールアドレスは既に登録されています',
        });
    }

    const hashedPassword = await bcrypt.hash(
        input.password,
        10,
    );
```

`bcrypt.hash` の第2引数の `10` は変換にかかる計算量を決めるコスト値です。繰り返す回数そのものではなく、`2` の `10` 乗という形で効きます。`11` に上げると計算量はおおよそ2倍、`12` ならおおよそ4倍です。1つ増やすだけで1回の変換にかかる時間が倍になり、盗まれたハッシュから元のパスワードを総当たりで探す側も、同じ倍率だけ待たされます。小さくすると登録とログインは速くなりますが同じ時間で総当たりで試される回数も増えます。大きくしすぎると登録とログインのたびに読者を待たせます。`10` はその折り合いとして広く使われている値です。

同じパスワードでも、保存される文字列は毎回違います。`bcrypt` が変換のたびに乱数を混ぜ、その乱数も結果の文字列へ含めるためです。だから同じ文字列が2つ並んでいるかを見ても同じパスワードを使っている人は分かりません。

確認ポイント:

- ☐ 重複メールを `CONFLICT` で弾いています。
- ☐ `bcrypt.hash` でパスワードをハッシュ化しています。

```
// filepath: src/server/api/routers/auth.ts(続き)
const user = await prisma.user.create({
  data: {
    email: input.email,
    name: input.name,
    password: hashedPassword,
    role: USER_ROLE.USER,
    isActive: true,
  },
});
```

このコードで一番大事な行は `role: USER_ROLE.USER` です。ここを `input.role` にして利用者の入力から受け取る作りになすると登録フォームへ `role: "ADMIN"` を混ぜて送るだけで、誰でも管理者アカウントを作れてしまいます。登録 API は誰でも呼べる `publicProcedure` なので権限だけは入力から切り離してサーバー側で固定します。 `isActive: true` も同じ考え方で、有効か無効かを利用者に決めさせません。利用者には、名前やメールアドレスなど、登録に必要な情報を入力してもらいます。一方、管理者かどうかはサーバー側で決めます。利用者が送った値を、そのまま操作権限として採用しないためです。入力の形式が正しくても、その利用者に指定させてよい値とは限りません。

Day 07: 認証バックエンドを作ってログインを動かそう

確認ポイント:

- ☐ 新規ユーザーを `USER_ROLE.USER` で作成しています。

```
// filepath: src/server/api/routers/auth.ts(続き)

const sessionUser: SessionUser = {
  id: user.id,
  email: user.email,
  role: user.role,
  version: user.sessionVersion,
};

await createSession(sessionUser);

return {
  user: {
    id: user.id,
    email: user.email,
    name: user.name,
    avatar: user.avatar,
    role: user.role,
  },
};
```

登録が完了した場合はここで戻り値が決まります。以下の `catch` は、事前検索の後に同じメールアドレスが登録された場合も同じエラーにそそえます。

```
// filepath: src/server/api/routers/auth.ts(続き)
    } catch (error) {
      if (error instanceof TRPCError) throw error;
      if (isUserEmailUniqueConstraintError(error)) {
        throw new TRPCError({
          code: 'CONFLICT',
          message:
            'このメールアドレスは既に登録されています',
        });
      }
      handleUnexpectedError('ユーザー登録処理', error);
    }
  },
},
```

登録が終わったら `createSession` で認証用 Cookie の保存を試みます。この処理が成功すると、そのユーザーとしてログインが必要な API を呼びます。Day 06 の登録画面はログイン画面へ移動するので、読者はそこでログイン操作も確認します。返す `user` から `password` を外してあるのはログインの戻り値とそろえたのと同じ理由です。`catch` の形もログインと合わせてあるので `TRPCError` はそのまま画面へ渡ります。DB の一意制約が同時登録を拒否したときはメール重複だけを `CONFLICT` へ変換し、それ以外は `handleUnexpectedError` へ渡します。

確認ポイント:

- 登録直後、`sessionVersion` を渡して `createSession` でログイン状態にしています。

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/routers/auth.ts(続き)
logout: publicProcedure.mutation(async () => {
  await deleteSession();
  return { success: true };
}),
```

logout は Cookie を消して本人のブラウザでログイン状態を終了させます。サーバー側にセッション一覧を保存する方式ではないため、この手続きが消すDBレコードはありません。別端末の Cookie を失効させる処理でもありません。

```
// filepath: src/server/api/routers/auth.ts(続き)
getSession: publicProcedure.query(async ({ ctx }) => {
  if (!ctx.session) return null;

  const user = await prisma.user.findUnique({
    where: { id: ctx.session.userId },
    select: { ...USER_DETAIL_SELECT, sessionVersion:
true },
  });

  if (!user || !user.isActive ||
user.sessionVersion !== ctx.session.version) {
    return null;
  }

  return {
    user: {
      id: user.id,
      email: user.email,
      name: user.name,
      avatar: user.avatar,
      role: user.role,
      isActive: user.isActive,
    },
  };
}),
```

Day 07: 認証バックエンドを作ってログインを動かそう

`getSession` が Cookie の中身をそのまま返さず、毎回データベースを引き直しているのには理由があります。Cookie に入っているのはログインした時点の名前や役割で、そのあと本人が名前を変えても古いままです。ここで取り直しておくで画面のどこに出しても最新の値になります。

```
!user || !user.isActive || user.sessionVersion !== ctx.session.version
```

は、退会・無効化や古い Cookie を見つけたときに `null` を返す条件です。トークン自体は期限まで有効なので、この確認が無いと無効化した利用者をログイン済みとして扱ってしまいます。戻り値の `user` には `sessionVersion` を含めず、表示に必要な情報だけを返します。

確認ポイント:

- ☐ `logout` は Cookie を削除しています。
- ☐ `getSession` は未ログイン、無効化、セッション版数不一致で `null` を返します。

```
// filepath: src/server/api/routers/auth.ts(続き)
getCurrentUser: protectedProcedure.query(
  async ({ ctx }) => {
    const user = await prisma.user.findUnique({
      where: { id: ctx.session.userId },
      select: {
        ...USER_DETAIL_SELECT,
        createdAt: true,
        updatedAt: true,
      },
    });
  });
```

5 つの API のうち、`protectedProcedure` で作るのは `getCurrentUser` だけです。未ログインなら `null` を返せばよい `getSession` と違ってこちらは本人しか呼ばない前提なので入口で弾いてしまうほうが安全です。

`ctx.session.userId` を鍵にしてデータベースを引き直すのはトークンを作ったあとに名前やアイコンが変わっていても最新の値を返すためです。`select` に `createdAt` と `updatedAt` を足しているのはプロフィール画面で登録日と更新日を出すからです。必要な項目だけを並べておけばパスワードのような返してはいけない列が紛れ込みません。

確認ポイント:

- ☐ `getCurrentUser` はログイン中ユーザーの詳細を取得しています。

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/routers/auth.ts(続き)

if (!user) {
  throw new TRPCError({
    code: 'NOT_FOUND',
    message: 'ユーザーが見つかりません',
  });
}

if (!user.isActive) {
  throw new TRPCError({
    code: 'FORBIDDEN',
    message: 'このアカウントは無効化されています',
  });
}

return user;
},
),
});
```

API	種別	認証	用途
login	mutation	不要	ログイン
register	mutation	不要	ユーザー登録
logout	mutation	不要	ログアウト
getSession	query	不要	現在のセッション確認 (null 許容)

API	種別	認証	用途
<code>getCurrentUser</code>	query	必須	ログインユーザーの詳細取得

`bcrypt.hash(password, 10)` の `10` はソルトラウンド。数字が大きいほど安全だが遅くなります。`10` が一般的なバランス。

確認ポイント:

- ☐ `src/server/api/routers/_helpers/select.ts` を教材のコードと照合しました。
- ☐ `src/server/api/routers/auth.ts` を教材のコードで作り直しました。
- ☐ `authRouter` に 5 つの API(login / register / logout / getSession / getCurrentUser)があります。

学んだこと: パスワードは平文で保存せず `bcrypt.hash` でハッシュ化し、照合は `bcrypt.compare` で行います。

Step 4: API を繋ぎ直す(ルーター登録 + HTTP ハンドラー・読む目安: 5分)

ゴール: 作った auth ルーターを tRPC に登録し、HTTP リクエストを受け付けられるようにします。

4-1. root.ts(ルーターを束ねる)

`src/server/api/root.ts` を開き、中身を教材のコードへ置き換えます。

```
// filepath: src/server/api/root.ts
import { authRouter } from './routers/auth';
import { createCallerFactory, createTRPCRouter } from
'./trpc';

export const appRouter = createTRPCRouter({
  auth: authRouter,
});

export type AppRouter = typeof appRouter;

export const createCaller = createCallerFactory(
appRouter);
```

`export type AppRouter = typeof appRouter;` の1行が画面側の型の出どころです。この型を書き出しておくと `api.auth.login.useMutation()` と打った時点で、引数の形も戻り値の形もエディタが知っている状態になります。サーバーの手続きを増やせば画面側の候補も自動で増えます。逆にこの行を消すと画面側は何を呼べるのか分からなくなり、すべて手で型を書くことになります。

プロジェクトやタスクのルーターはそれを実際に使う Day 09 以降で 1 つずつ足していきます。

4-2. route.ts(HTTP ハンドラー)

`src/app/api/trpc/[trpc]/route.ts` を開き、中身を教材のコードへ置き換えます。

```
// filepath: src/app/api/trpc/[trpc]/route.ts
import { fetchRequestHandler } from
  '@trpc/server/adapters/fetch';
import type { NextRequest } from 'next/server';
import { appRouter } from '@server/api/root';
import { createTRPCContext } from '@server/api/trpc';

const handler = (req: NextRequest) =>
  fetchRequestHandler({
    endpoint: '/api/trpc',
    req,
    router: appRouter,
    createContext: () =>
      createTRPCContext({ headers: req.headers }),
  });

export { handler as GET, handler as POST };
```

フォルダ名の `[trpc]` が角括弧で囲まれているのはURL のその部分に何が来ても受け取るという印です。おかげで `/api/trpc/auth.login` でも `/api/trpc/auth.getSession` でも、呼ばれるのはこの 1 ファイルになります。API を 1 つ足すたびにファイルを作らずに済むのは入口をここへまとめているからです。

確認ポイント:

- ☐ `src/server/api/root.ts` を教材のコードで作り直しました。

Day 07: 認証バックエンドを作ってログインを動かそう

- ☐ `src/app/api/trpc/[trpc]/route.ts` を教材のコードで作り直しました。
- ☐ ここまでのコードの綴りと括弧の対応を確認しました。

Step 5: middleware.ts を作る(ルート保護・読む目安: 8分)

ゴール: ログインしていないユーザーを自動でログイン画面にリダイレクトする仕組みを作ります。

このファイルは Next.js の Edge Runtime で動きます。 `config.matcher` で指定した対象ルートの「入口」で、Cookie に有効な JWT があるかを確認します。

`src/middleware.ts` を新規作成します。置き場所は `src/app/` の中ではなく、`src/` の直下です。

```
// filepath: src/middleware.ts
import { jwtVerify } from 'jose/jwt/verify';
import { type NextRequest, NextResponse } from
'next/server';
import { isValidRedirectUrl } from '@lib/redirect';

const COOKIE_NAME = 'session';

const PUBLIC_PATHS = ['/login', '/register'];
```

`COOKIE_NAME` を `session.ts` から取り込まず同じ値をここへ書き直しています。middleware は Edge Runtime で動くため `next/headers` の `cookies()` を使うファイルを読み込めないからです。2 か所の値がずれると

middleware が Cookie を見つけられず、ログイン済みの人まで全員ログイン画面へ戻されます。片方を変えるときは必ずもう片方も直してください。

`PUBLIC_PATHS` に並べたパスはログイン前でも開ける入口です。うっかりここへ `/dashboard` を書き足すとそのページだけ認証を通さず中身が見えてしまいます。

確認ポイント:

- ☐ Cookie 名と公開パスを定数で定義しています。

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/middleware.ts(続き)
function isPublicPath(pathname: string): boolean {
  return PUBLIC_PATHS.some(
    (path) =>
      pathname === path
      || pathname.startsWith(`${path}/`),
  );
}

function isValidCallbackPath(path: string): boolean {
  return (
    isValidRedirectUrl(path)
    && !path.includes('/://')
  );
}

function getJwtSecret(): Uint8Array {
  const secret = process.env['JWT_SECRET'];
  if (!secret) {
    throw new Error('JWT_SECRET is not set');
  }
  return new TextEncoder().encode(secret);
}
```

`isPublicPath` が `pathname === path` だけでなく `startsWith( ${path}/ )` も見ているのは `/login/reset` のような下の階層まで公開にするためです。前方一致だけにとすると `/loginx` のような別のパスまで公開になります。

`getJwtSecret` が `process.env` を毎回読み直しているのはミドルウェアが Edge Runtime で動くためです。ここでは `src/lib/env.ts` のような読み込み済みの設定を使えません。値が無いときにその場で例外を投げているのは鍵が無いまま検証を続けるとどのトークンも通らない画面になり、原因が分からなくなるからです。

確認ポイント:

- ☐ 公開パス判定と callbackUrl 検証の helper があります。
- ☐ `JWT_SECRET` を Edge Runtime でも読める形にしています。

```
// filepath: src/middleware.ts(続き)
export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // 公開ページはスキップ
  if (isPublicPath(pathname)) {
    return NextResponse.next();
  }

  // tRPC エンドポイントはスキップ
  // (tRPC 層の protectedProcedure で認証を担保)
  if (pathname.startsWith('/api/trpc')) {
    return NextResponse.next();
  }
}
```

Day 07: 認証バックエンドを作ってログインを動かそう

`/api/trpc` をここで通しているのは認証をしないという意味ではありません。API 側は `protectedProcedure` が1件ずつ判定しているので二重に止める必要がないからです。ここで止めてしまうと未ログインでも呼べるはずの `login` 自体が呼べなくなり、ログインできない画面になります。

確認ポイント:

- ☐ `/login` / `/register` / `/api/trpc` は middleware で通しています。

```
// filepath: src/middleware.ts(続き)
const token = request.cookies.get(COOKIE_NAME)?.value;

// Cookie なし → ログインへリダイレクト
if (!token) {
  const loginUrl = new URL('/login', request.url);
  loginUrl.searchParams.set(
    'callbackUrl',
    isValidCallbackPath(pathname)
      ? pathname
      : '/dashboard',
  );
  return NextResponse.redirect(loginUrl);
}
```


`callbackUrl` に開こうとしたパスを入れているのはログインし終わった読者を元の場所へ戻すためです。これが無いとどのページから飛ばされても行き先はダッシュボード1つになり、作業の途中で追い出された人は自分で戻り道を探すことになります。

検証に落ちたときの行き先を `/dashboard` にしているのは危ない値をそのまま持ち回らないためです。ここで元の値を残すとあとの処理でうっかり使われる余地が残ります。

確認ポイント:

- ☐ Cookie が無いときに `/login` へ戻しています。
- ☐ `callbackUrl` に外部 URL を入れない検証をしています。

```
// filepath: src/middleware.ts(続き)
// JWT 検証
const jwtSecret = getJwtSecret();
try {
  await jwtVerify(token, jwtSecret, {
    algorithms: ['HS256'],
  });
  return NextResponse.next();
} catch {
  // 無効なトークン: Cookie 削除してログインへ
  const loginUrl = new URL('/login', request.url);
  const response = NextResponse.redirect(loginUrl);
  response.cookies.delete(COOKIE_NAME);
  return response;
}
```

鍵の取得を `try` の中に置くと、設定漏れの例外まで `catch` が受け取り、Cookie を削除してログイン画面へ戻します。設定漏れの原因が表示されないので、`getJwtSecret()` は `try` の前で呼びます。`JWT_SECRET is not set` が出たら、ローカルでは `.env`、公開先では Vercel の Environment Variables を確認して鍵を設定します。ローカルの開発サーバーを再起動し、公開先では再デプロイしてから開き直してください。

`jwtVerify` が確かめるのは署名が秘密鍵と合っているかと、`exp` の期限が切れていないかの 2 点です。どちらかが崩れていれば例外になり、処理は `catch` 側へ落ちます。そこで Cookie を削除してからログイン画面へ送るのは壊れたトークンを持ったままリダイレクトを繰り返す状態を避けるためです。署名を見ずに中身のデコー

ドだけで通す作りにすると期限切れのトークンでも、自作のトークンでも入れてしまいます。middleware が守っているのはページの表示だけで、API 側の守りは Step 2 の `protectedProcedure` が受け持ちます。

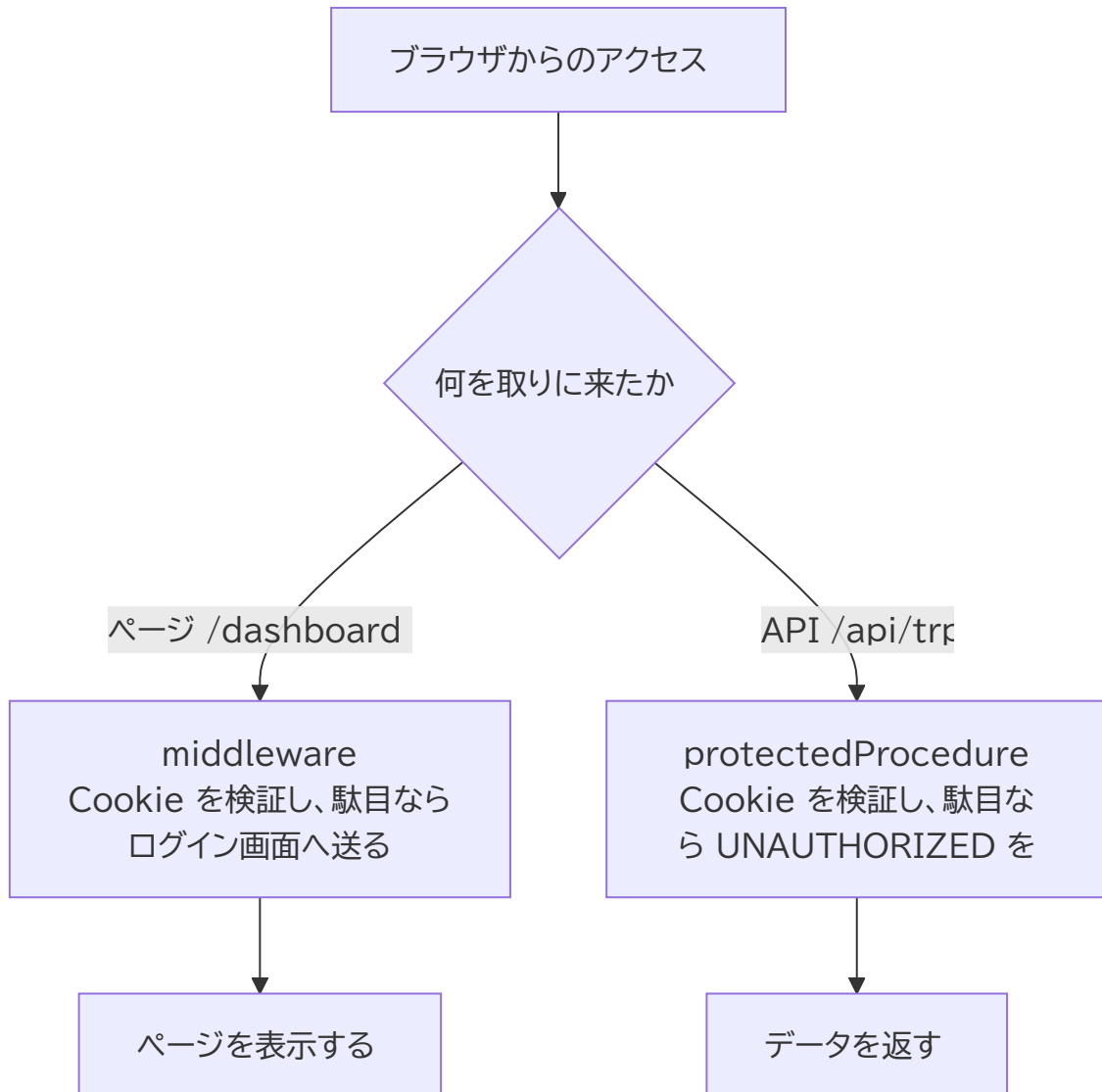


図5: Step 5: middleware.ts を作る

守りは2枚あり、担当する入口が違います。middleware を通らずに API を直接叩く経路があるため片方だけでは足りません。ページは見えないのにデータは取れる、という穴を防ぐのが右側の枝です。

確認ポイント:

- ☐ JWT が有効なら通し、無効なら Cookie を削除しています。

```
// filepath: src/middleware.ts(続き)
export const config = {
  matcher: [
    '/((?!_next/static|_next/image|favicon.ico).*)',
  ],
};
```

`config.matcher` は middleware を動かす URL を絞り込む指定です。

`_next/static` と `_next/image`、`favicon.ico` を外しているのは画像 1 枚を読むたびに JWT の検証を走らせても得るものが無く、表示が遅くなるだけだからです。`_next/image` は Next.js が変換して配る画像の置き場で、`public/` に自分で置いた画像はここに入りません。`(?!...)` は「ここに並べた語で始まらない URL にだけ一致する」という書き方です。つまりここへ並べたパスは middleware を通らず、JWT の検証を受けません。並べる数を増やしすぎると守りたいページまで middleware を素通りします。足すのは画像やスタイルのような表示用ファイルだけにとどめます。

この書き方は「除外したもの以外はすべて」なのでトップページの `/` も対象に入ります。今日からログインしていない状態で `/` を開くとログイン画面へ送られます。

Day 01 で作ったあの画面はログインしたあとでないと見られなくなります。壊したわけではありません。次の Step 6 で動作を確認するとき `/` が開かないのは正常な結果です。

コラム: middleware は「どこ」で動くのか

matcher に当てはまった URL へのリクエストがmiddlewareを通ります。ただし通ったものの全部がJWTの検証まで進むわけではありません。`/login` と `/register`、`/api/trpc` は手前で通します。Cookie が無いリクエストは検証の前に `/login` へ送ります。検証まで進むのはCookieを持った保護対象のリクエストだけです。

それでも体感の速さは変わります。middleware を置く場所がデプロイ先によって変わるためです。

Day 30 で使う Vercel は middleware を世界中の拠点へ配ります。通常はアクセス元に近い拠点が判定を受け持ちます。未ログインのアクセスは、アプリ本体へ届く前に `/login` へ送り返されます。

サーバーを1台だけ立てる場合はどうでしょうか。`next start` で動かすと middleware はそのサーバーの中だけに置かれます。置き場所がアメリカなら東京の人は毎回そこまで往復してから `/login` へ送られます。往復の分だけ待たされます。

ここで誤解しやすいのは速さの理由をひとつに決めてしまうことです。理由は2つあります。1つは実行環境そのものです。Edge Runtime は使える機能を絞った軽い環境なので立ち上げ直すときの待ち時間が短く済みます。効くのは立ち上げ直す場面だけで、すでに立ち上がっている環境が受けるリクエストの速さには関係しません。もう1つが距離です。Next.js の公式ドキュメントはこの実行環境について「edge で動かすことを必須とせず、単一リージョン(1か所の地域だけで処理する構成)のサーバーでも動く」と書いています。つまり実行環境の軽さと、拠点をどこに置くかは別の話です。

ところが手元の `npm run dev` では話が変わります。ここでも middleware は Edge Runtime で動きますが本物ではありません。Next.js は Node.js の中に JavaScript のサンドボックス(外と切り離れた実行用の領域)を用意し、そこで Edge Runtime の振る舞いを真似させています。Docker のコンテナとは別物で、1つの Node.js プロセスの中の話です。公式ドキュメントはこの真似る処理そのものに時間がかかると書いています。middleware を1回通るたびに、このサンドボックスへ出入りする処理時間がかかります。

手元でだけ増えている手間はここです。ただし JWT の検証とどちらが重いかはこの教材では計っていません。公式ドキュメントもそこまでは書いていません。言えるのは本番には無い手間が1つ余分に挟まっている、というところまでです。Day 30 で公開する Vercel では拠点に置かれた実行環境がそのまま動くのでこの分はかかりません。

回数を決めているのが matcher です。ここで取り違えやすいのが保存すると画面が自動で切り替わる仕組み(Fast Refresh)の扱いです。この仕組みは WebSocket(一度つないだ通信路を開いたまま使い回す仕組み)で動きます。つなぎ先の `/_next/webpack-hmr` は matcher の除外に入っていないんですが Next.js はこの接続を通常の経路より手前で受け取ります。matcher の書き方に関係なく、この通信は middleware に届きません。保存1回につき middleware が1回増える、という話ではないわけです。

増えるのは別のところですよ。サーバー側で動くファイルを保存すると Next.js はいま開いている画面をその場で取り直します。この取り直しは今いる URL へのリクエストなので matcher に当たり middleware を通ります。エラーが出たときに開発ツ-

ルが投げる `/__nextjs_original-stack-frames` も同じです。どちらも matcher から外していないので保存とエラーを繰り返すほどサンドボックスへの出入りは増えていきます。

開発中だけ重いと感じたときはまず matcher が何を通していているか確かめてください。本番では飛ばないリクエストなので公開したあとの速さには影響しません。

matcher を絞る話も、ここにつながります。当てはまる URL が多いほどこの判定を通る回数も増えます。ただし外してあるのは `_next/static` と `_next/image`、`favicon.ico` の3つだけです。`public/` に置いた画像やスタイルは外れていないのでいまの書き方でも middleware を通ります。

middleware の判定フロー:

Day 07: 認証バックエンドを作ってログインを動かそう

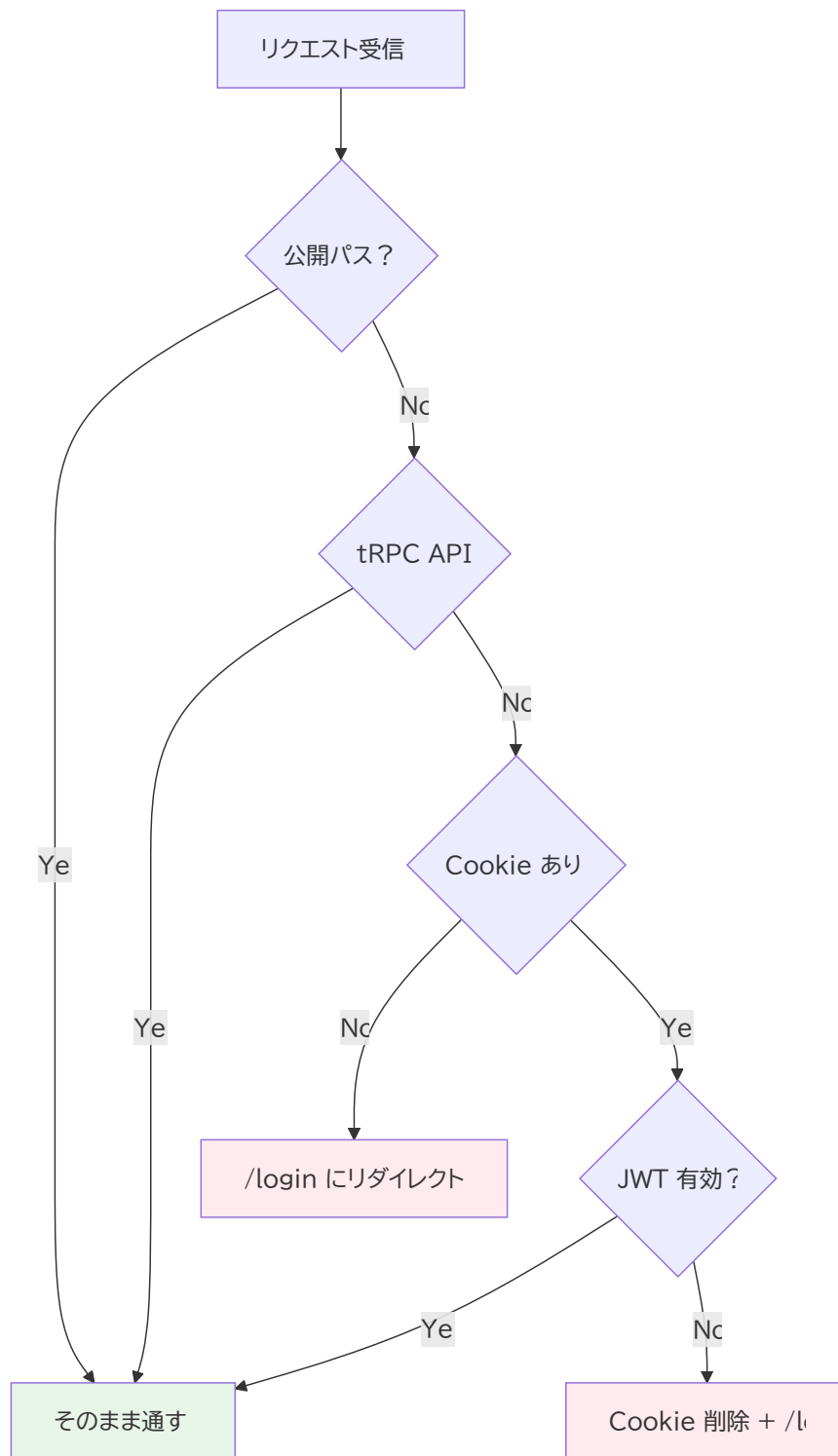


図 6: コラム: middleware は「どこ」で動くのか

なぜ middleware で `session.ts` を import しない? middleware は Edge Runtime で動くため `cookies()` を使う `session.ts` は import できません。 `jose` を直接使って JWT 検証します。

`isValidCallbackPath` の役割: `callbackUrl` に外部 URL を仕込む Open Redirect 攻撃を防ぎます。判定の本体は Day 05 で作った `src/lib/redirect.ts` の `isValidRedirectUrl` です。 `/` で始まること、 `//` では始まらないこと、 `\` やタブ・改行・復帰を含まないことを、そちらが確かめます。ここで足しているのは `://` を含まないという条件だけです。同じ規則を 2 か所へ書き写すと片方だけ直したときに緩いほうが残ります。

確認ポイント:

- ☐ `src/middleware.ts` が作成できました(`src/app/` ではなく `src/` 直下)。
- ☐ `config.matcher` でアセットファイルを除外しています。

学んだこと: Next.js の middleware は `config.matcher` で指定したルートの入口で動きます。認証チェックを 1 か所に集約できるのでページごとにチェックコードを書く必要がありません。

Step 6: ログインして動作確認する(読む目安: 5分)

ゴール: ここまで作った認証バックエンドが実際に動くことを確認します。

開発サーバーを起動します。

Day 07: 認証バックエンドを作ってログインを動かそう

```
npm run dev
```

Docker Desktop を開き、Day 01 で作った PostgreSQL のコンテナが起動中であることを確認します。ログイン画面を表示するだけではDB のスキーマや初期データの有無は分かりません。この Step では後述の初期アカウントでログインできるところまで確かめます。

Day 01 のスキーマ反映や初期データ投入が未完了の場合は次の2つを実行してください。原因が分からないエラーには先に付録のトラブルシューティングを使います。

```
npm run db:push  
npm run db:seed
```

`npm run db:seed` は初期データの2つのプロジェクトを削除して作り直します。実行すると消える対象の一覧が表示され **続行しますか? [y/N]:** と聞かれるので **y** を入力して Enter を押します。Day 07 の時点では消えて困るものはまだありません。Day 06 で登録した自分のアカウントも残ります。Day 09 以降、初期データの2つのプロジェクトの中にタスクやコメントを足したあとは同じ操作でそれらが消えます。自分で新しく作ったプロジェクトは消えません。

ブラウザで `http://localhost:3000/login` を開きます。冒頭に載せた画面と同じものが出ます。

seed データのログイン情報:

メール	パスワード	権限	名前
admin@example.com	password123	管理者	管理者

メール	パスワード	権限	名前
user1@example.com	password123	一般ユーザー	田中太郎
user2@example.com	password123	一般ユーザー	山田花子
empty@example.com	password123	一般ユーザー	新人 太郎

普段は1行目の管理者アカウントを使います。残り3つは権限の違いを試すときに使います。たとえば「管理者だけに見える画面」を確かめるときは

user1@example.com でログインし直します。

1. メールとパスワードを入力してログインボタンを押します。
2. 「おかえりなさい、管理者さん」トーストが表示されます。
3. ダッシュボードに遷移します。



図 7: ダッシュボード。Personal Message のカードに大きな見出しが出て下に OWNER・TODAY・NEXT の3枚のカードが並んでいる

Day 07: 認証バックエンドを作ってログインを動かそう

ダッシュボードの中身は Day 02 で作ったままです。今日変わったのはログインを通らないところへ来られなくなったことです。

書き直した登録処理も確認する

ログインだけでは、`register` がユーザーを保存できるかは確認できません。Day 06 の画面から、いま書き直した登録処理も試します。管理者でログインしたウィンドウは開いたままにしておきます。

1. 別のシークレットウィンドウを開き、`http://localhost:3000/register` を表示します。
2. 名前に `確認ユーザー`、メールに `day07-check@example.com` を入力します。パスワードと確認欄には、どちらも `ReaderDay07!` を入力します。
3. 「登録」を押します。ログイン画面へ移動し、「登録が完了しました」が表示されることを確認します。
4. 登録したメールアドレスとパスワードでログインし、ダッシュボードへ移動できることを確認します。

メール重複のエラーが出たら、以前の確認でそのアドレスを登録しています。

`day07-check2@example.com` のように、まだ登録していないアドレスに変えて試してください。それ以外のエラーは、画面の文言とターミナルのログを確認し、このあとにある「つまずきポイント」で原因を調べます。

確認が終わったらシークレットウィンドウを閉じ、管理者でログインしたウィンドウへ戻ります。次の Step 7 では、そのログイン時に保存された Cookie を確認します。

確認ポイント:

- ☐ `npm run dev` でエラーが出ません。

- ☐ ログインが成功してトーストが表示されます。
- ☐ ダッシュボードに遷移します。
- ☐ 新しいアカウントを登録し、そのメールアドレスでログインできました。

うまくいかないとき: ターミナルのエラーメッセージを確認します。よくある原因は「つまずきポイント」セクションにまとめてあります。

Step 7: DevTools で JWT と Cookie を確認する(読む目安: 5分)

ゴール: ブラウザに保存された JWT トークンの中身を確認し、認証ガードの動作を体験します。

7-1. Cookie を確認する

1. DevTools を開きます(`F12` または `Cmd+Option+I`)。
2. **Application** タブ → 左メニューの **Cookies** →
`http://localhost:3000`
3. `session` という名前の Cookie を見つけます。

確認項目	期待値
Name	<code>session</code>
HttpOnly	チェックあり
SameSite	<code>Strict</code>
Path	<code>/</code>

7-2. JWT をデコードする

開発環境の JWT もログインに使える認証情報です。外部サイトには貼り付けず、いま開いているブラウザの DevTools 内だけで中身を読みます。

1. `session` Cookie の値(長い文字列)をコピーします。
2. DevTools の **Sources** タブ → **Snippets** → **New snippet** を開きます。
3. Snippet に `decode-jwt-local` と名前を付け、次のコードだけを貼り付けます。
4. `Ctrl+Enter` または `Cmd+Enter` で実行し、表示された入力欄へ Cookie の値を貼り付けます。

Console に貼り付け制限の警告が出ても、制限を解除する文言は入力しません。Snippets の編集欄を使えば、警告を無効にせずコードを実行できます。

```
((() => {  
  const token =  
window.prompt('session Cookie の値を貼り付けてください')?.trim();  
  if (!token) throw new Error('JWT が入力されていません');  
  const parts = token.split('.');  
  if (parts.length !== 3 || !parts[2]) {  
    throw new Error('JWT は3つの部分で構成されます');  
  }  
  const base64 = parts[1].replace(/-/g,  
'+'').replace(/_/g, '/');  
  const padded = base64.padEnd(Math.ceil(base64.length /  
4) * 4, '=');  
  const bytes = Uint8Array.from(  
    atob(padded),  
    (character) => character.charCodeAt(0),  
  );  
  const payload = JSON.parse(new TextDecoder().decode(  
bytes));  
  const [header, , signature] = parts;  
  const replacement = signature[0] === 'A' ? 'B' : 'A';  
  const tamperedToken = `${header}.${parts[1]}.${{  
replacement}${signature.slice(1)}}`;  
  console.log({ payload, tamperedToken });  
})();
```

Day 07: 認証バックエンドを作ってログインを動かそう

JWT は `.` で区切られた 3 つの部分からできていて、2 つ目が Payload です。Payload は Base64URL という形なので `-` と `_` を置き換え、不足する `=` を足してから読みます。`TextDecoder` を通すのはメールアドレスや名前に日本語が入っても UTF-8 の文字として戻すためです。

Console に表示された `payload` を開くと次のフィールドを確認できます。

フィールド	表示例	意味
<code>userId</code>	<code>"cm..."</code>	ユーザー ID
<code>email</code>	<code>"admin@example.com"</code>	メールアドレス
<code>role</code>	<code>"ADMIN"</code>	権限
<code>iat</code>	発行時刻を表す数値	トークンの発行時刻 (UNIX 時間)
<code>exp</code>	発行時刻から 7 日後を表す数値	トークンの有効期限 (UNIX 時間)

UNIX 時間: ブラウザのコンソールで `new Date(1234567890 * 1000)` と入力すると人間が読める日時に変換できます。

このコードが行うのは Payload のデコードだけです。署名が正しいか、有効期限内かは確認していません。ログイン済みと判断するのはサーバー側の `jwtVerify` が秘密鍵を使って署名と期限を検証したあとだけです。

7-3. 署名を変えて認証ガードを体感する

1. Snippet の出力にある `tamperedToken` の値をコピーします。

2. Application → Cookies → `session` の Value を `tamperedToken` へ置き換えます。
3. ブラウザで `/dashboard` を再読み込みします。
4. 自動的に `/login` にリダイレクトされ、`session` Cookie が削除されます。

`tamperedToken` は署名部分の先頭を、`A` なら `B`、それ以外なら `A` へ必ず変更した値です。Payload は同じでも署名のバイト列が変わるため、Step 5 で作った middleware の `jwtVerify` が拒否します。Cookie を削除するのも middleware の `catch` に書いた処理です。

5. もう一度ログインするとダッシュボードが表示されます。

確認ポイント:

- ☐ `session` Cookie が存在します。
- ☐ DevTools 内だけで `userId`, `email`, `role`, `exp` が確認できました。
- ☐ 署名を変えたあとに `/dashboard` が表示できなくなり、Cookie が削除されました。
- ☐ 再ログインでダッシュボードが復活しました。

学んだこと: この教材では署名付き JWT を使います。中身のデコードは秘密鍵なしでできますが、それだけではログイン済みと判断できません。サーバーが署名と期限を検証して初めて認証に使えます。JWT には暗号化する形式もあります。今回の署名と暗号化の役割を分けて覚えましょう。

Pro パターンで書こう(認証ガードは early return で道順を見せる)

Step 5 の middleware は「公開パス → tRPC → Cookie なし → JWT 無効」と early return で判定を重ねています。この書き方を他のコードでも使ってみましょう。

Before(改善前のコード)

```
export function AuthGuard({
  authStatus,
  children,
}): {
  authStatus: 'loading' | 'guest' | 'authenticated';
  children: React.ReactNode;
}) {
  return authStatus === 'loading' ? (
    <main>セッション確認中</main>
  ) : authStatus === 'guest' ? (
    <main>ログイン画面へ移動します</main>
  ) : (
    <>{children}</>
  );
}
```

問題点: 条件が増えるほど `?` と `:` の対応を目で追う必要が出ます。3 つ目の状態を足すときはすでにある入れ子のどこへ差し込むかを考えることになります。認証ガードはあとから「メール未確認」「権限不足」といった分岐が増えやすい場所です。分岐を足すたびに全体を読み直す形だと差し込む位置を 1 つ間違えただけで、通してはいけない相手を通してしまいます。

After(プロが書くコード)

```
export function AuthGuard({
  authStatus,
  children,
}): {
  authStatus: 'loading' | 'guest' | 'authenticated';
  children: React.ReactNode;
}) {
  if (authStatus === 'loading') {
    return <main>セッション確認中</main>;
  }

  if (authStatus === 'guest') {
    return <main>ログイン画面へ移動します</main>;
  }

  return <>{children}</>;
}
```

強み: 判定の順番が上から下へそのまま読めます。新しいガード条件を足すときも、`if` と `return` を 1 組増やすだけで済みます。

覚えておきたいエッセンス: 認証ガードは分岐が増えやすい場所。三項演算子で詰め込むより先に返して本筋を残すほうが読みやすく育てやすいです。

完成コード全体

今日扱った10資産を以下にまとめます。`schema.prisma` は確認対象で、残り9ファイルが書き換え対象です。断片を貼り重ねる作業が続いたので途中でどこへ貼ったか分からなくなった場合は以下のコードを上から順に貼り付けて各ファイルを置き換えてください。1つのファイルが複数のブロックに分かれている場合はそのファイルの見出しの下にあるブロックを、出てくる順につなげたものが全文です。

ファイル	役割	対応する Step
<code>prisma/schema.prisma</code>	scaffold が配布したセッション版数の列を確認	Step 0
<code>src/lib/password.ts</code>	bcrypt で共通に使うパスワード検証	Step 0
<code>src/server/api/routers/_helpers/user-email-conflict.ts</code>	メール重複の識別	Step 0
<code>src/lib/session.ts</code>	JWT の発行・検証と Cookie の出し入れ	Step 1
<code>src/server/api/trpc.ts</code>	tRPC の土台と3種類の入口	Step 2

ファイル	役割	対応する Step
<code>src/server/api/routers/_helpers/select.ts</code>	Prisma で取り出す列の定型	Step 3
<code>src/server/api/routers/auth.ts</code>	ログイン・登録・ログアウトの手続き	Step 3
<code>src/server/api/root.ts</code>	手続きの一覧表	Step 4
<code>src/app/api/trpc/[trpc]/route.ts</code>	HTTP から tRPC への橋渡し	Step 4
<code>src/middleware.ts</code>	ログインしていない人をログイン画面へ送る	Step 5

実際に書き換えるのは9ファイルです。`prisma/schema.prisma` は書き換えず、Day 01 の配布値を確認するために表へ含めています。

`prisma/schema.prisma`

User モデルに次の1行が1本だけあることを確認します。ここは確認用で、追加用のコードではありません。

```
// 読み比べ用: prisma/schema.prisma の User モデルにある1行
sessionVersion Int @default(0) @map("session_version")
```

`src/lib/password.ts`

次がこのファイルの全文です。Step 0 と同じく、既存の中身をすべて削除してから貼り付けます。

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/lib/password.ts
import { z } from 'zod';

export const BCRYPT_PASSWORD_MAX_BYTES = 72;

export const isPasswordWithinBcryptLimit = (password:
string): boolean =>
  new TextEncoder().encode(password).byteLength <=
BCRYPT_PASSWORD_MAX_BYTES;

export const createPasswordSchema = (
minimumLengthMessage: string) =>
  z
    .string()
    .min(8, minimumLengthMessage)
    .regex(/[A-Z]/,
'パスワードには大文字を含める必要があります')
    .regex(/[a-z]/,
'パスワードには小文字を含める必要があります')
    .regex(/[0-9]/,
'パスワードには数字を含める必要があります')
    .regex(/^[A-Za-z0-9]/,
'パスワードには特殊文字を含める必要があります')
    .refine(isPasswordWithinBcryptLimit,
'パスワードはUTF-8で72バイト以内にしてください');
```

`src/server/api/routers/_helpers/user-email-conflict.ts`

次がこのファイルの全文です。`P2002` というコードだけで決めず、`User.email` の制約かまで確かめます。

```
// filepath: src/server/api/routers/_helpers/user-email-conflict.ts
import { Prisma } from '@prisma/client';

export const isUserEmailUniqueConstraintError = (error:
unknown): boolean => {
  if (!(error instanceof
Prisma.PrismaClientKnownRequestError) || error.code !==
'P2002') {
    return false;
  }

  const target = error.meta?.['target'];
  const modelName = error.meta?.['modelName'];
  return (
    Array.isArray(target) &&
    target.length === 1 &&
    target[0] === 'email' &&
    (modelName === undefined || modelName === 'User')
  );
};
```

`target` を配列として1項目の `email` に限定し、`modelName` も `User` かを確かめます。この条件が無いと、別のテーブルや別の列の一意制約エラーまで「メールアドレスが重複している」と誤って案内してしまいます。

src/lib/session.ts

インポートと鍵の組み立て:

```
// filepath: src/lib/session.ts
// 完成版: インポートと鍵の組み立て
import { type JWPayload, jwtVerify, SignJWT } from
'jose';
import { cookies } from 'next/headers';
import { USER_ROLE, type UserRole } from
'./constant/roles';
import { env } from './env';

function getKey(): Uint8Array {
  const SECRET_KEY = env.JWT_SECRET;
  const encoded = new TextEncoder().encode(SECRET_KEY);
  return new Uint8Array(encoded);
}
```

1枚目は jose と Cookie 操作の取り込み、そして署名に使う鍵を組み立てる部分です。`getKey` を関数にしてあるのは `.env` の `JWT_SECRET` を読む時点と呼び出しの瞬間まで遅らせるためです。ファイルを読み込んだ瞬間に鍵を確定させると環境変数がまだ入っていない場面で起動そのものが止まります。

型定義と定数:


```
// filepath: src/lib/session.ts
// 完成版: 型定義と定数
export interface SessionPayload {
  userId: string;
  email: string;
  role: UserRole;
  version: number;
  exp: number;
}

export interface SessionUser {
  id: string;
  email: string;
  role: UserRole;
  version: number;
}

const POSTGRES_INTEGER_MAX = 2_147_483_647;
const COOKIE_NAME = 'session';
const COOKIE_MAX_AGE = 60 * 60 * 24 * 7; // 7日間
```

トークンに詰める中身が `SessionPayload`、アプリ側から渡す材料が `SessionUser` です。2つを分けてあるのでトークンの有効期限を表す `exp` を画面側が組み立てる必要はありません。`COOKIE_MAX_AGE` は秒で数えるため7日間を `60 * 60 * 24 * 7` と書いてあります。

ペイロード検証と読み取り:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/lib/session.ts(完成版: ペイロード検証と読み取り)

function readSessionPayload(payload: JWTPayload):
SessionPayload | null {
  if (
    typeof payload['userId'] !== 'string' ||
    typeof payload['email'] !== 'string' ||
    (payload['role'] !== USER_ROLE.USER &&
payload['role'] !== USER_ROLE.ADMIN) ||
    typeof payload['exp'] !== 'number'
  ) return null;

  const rawVersion = Object.hasOwn(payload, 'version') ?
payload['version'] : 0;
  const isValidVersion =
    typeof rawVersion === 'number' &&
    Number.isInteger(rawVersion) &&
    rawVersion >= 0 &&
    rawVersion <= POSTGRES_INTEGER_MAX;
  if (!isValidVersion) return null;

  return {
    userId: payload['userId'],
    email: payload['email'],
    role: payload['role'],
    version: rawVersion,
    exp: payload['exp'],
  };
}
```

`jwtVerify` が返す中身は署名が正しくてもクレームの型まで保証されていません。そこで各項目が期待した型で入っているかを確認、新規トークン必須の `version` を検証します。旧トークンでプロパティ欠落の場合のみ `0` を採用し、異常クレームは `null` で弾きます。

signSessionToken — トークンの発行:

```
// filepath: src/lib/session.ts
// 完成版: signSessionToken — トークンの発行
export async function signSessionToken(
  payload: SessionPayload,
): Promise<string> {
  const jwtPayload: Record<string, unknown> = {
    userId: payload.userId,
    email: payload.email,
    role: payload.role,
    version: payload.version,
    exp: payload.exp,
  };

  return await new SignJWT(jwtPayload)
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('7d')
    .sign(getKey());
}
```

Day 07: 認証バックエンドを作ってログインを動かそう

`SignJWT` は `Record<string, unknown>` の形を求めるためいったん詰め替えてから渡します。`setProtectedHeader` で HS256 を宣言し、`setIssuedAt` で発行時刻、`setExpirationTime` で7日後の期限を書き込みます。最後の `sign` で鍵を使った署名が付き、文字列のトークンになります。

verifySessionToken — トークンの検証:

```
// filepath: src/lib/session.ts
// 完成版: verifySessionToken - トークンの検証
export async function verifySessionToken(
  token: string,
): Promise<SessionPayload | null> {
  try {
    const { payload } = await jwtVerify(token, getKey(), {
      algorithms: ['HS256'],
    });

    const session = readSessionPayload(payload);
    if (!session) {
      console.error('Invalid session payload structure');
      return null;
    }

    return session;
  } catch {
    console.error('Failed to verify session token');
    return null;
  }
}
```

`jwtVerify` は署名が合わなければ例外を投げます。 `algorithms: ['HS256']` を明示してあるのは攻撃者が「署名なし」を意味する別のアルゴリズムを名乗って検証をすり抜ける手口を塞ぐためです。読めなかった場合は例外を外へ流さず `null` を返し、呼び出し側では未ログインとして扱います。

Cookie への保存:

```
// filepath: src/lib/session.ts
// 完成版: Cookie への保存
export async function saveSessionCookie(
  token: string,
): Promise<void> {
  const cookieStore = await cookies();
  cookieStore.set(COOKIE_NAME, token, {
    httpOnly: true,
    secure:
      process.env['NODE_ENV'] === 'production'
      && process.env['PLAYWRIGHT_TEST'] !== '1',
    sameSite: 'strict',
    maxAge: COOKIE_MAX_AGE,
    path: '/',
  });
}
```

`httpOnly: true` を付けると JavaScript から Cookie を読めなくなり、ページへ紛れ込んだスクリプトにトークンを盗まれる経路が消えます。

`sameSite: 'strict'` は別サイトからの遷移で Cookie を送らない指定です。

`secure` を本番だけ有効にしてあるのは開発中の `http://localhost` でも Cookie を保存できるようにするためです。

createSession — 発行と保存の連結:

```
// filepath: src/lib/session.ts
// 完成版: createSession - 発行と保存の連結
export async function createSession(
  user: SessionUser,
): Promise<string> {
  const expiresAt =
    Math.floor(Date.now() / 1000) + COOKIE_MAX_AGE;
  const payload: SessionPayload = {
    userId: user.id,
    email: user.email,
    role: user.role,
    version: user.version,
    exp: expiresAt,
  };

  const token = await signSessionToken(payload);
  await saveSessionCookie(token);

  return token;
}
```

ログイン処理から呼ぶ入口です。期限を計算し、トークンを作り、Cookie に保存する3つを1本にまとめてあります。呼び出す側は `createSession(sessionUser)` の1行で済み、有効期限の秒数や Cookie の名前を知らなくても動きます。

getSession と deleteSession:

```
// filepath: src/lib/session.ts
// 完成版: getSession と deleteSession
export async function getSession():
Promise<SessionPayload | null> {
  const cookieStore = await cookies();
  const token = cookieStore.get(COOKIE_NAME)?.value;

  if (!token) {
    return null;
  }

  return await verifySessionToken(token);
}
export async function deleteSession(): Promise<void> {
  const cookieStore = await cookies();
  cookieStore.delete(COOKIE_NAME);
}
```

`getSession` は Cookie を取り出して `verifySessionToken` に渡すだけの薄い関数です。Cookie が無ければ署名の検証を試さずに `null` を返します。

`deleteSession` は Cookie を消すだけで、サーバー側に消すべき記録はありません。ログアウトが1行で終わるのは状態をトークンの中に持たせているからです。

verifySession — 画面向けの形へ変換:


```
// filepath: src/lib/session.ts
// 完成版: verifySession - 画面向けの形へ変換
export async function verifySession():
Promise<SessionUser | null> {
  const session = await getSession();

  if (!session) {
    return null;
  }

  return {
    id: session.userId,
    email: session.email,
    role: session.role,
    version: session.version,
  };
}
```

トークンの中身は `userId` という名前ですがアプリ側で扱うユーザーは `id` という名前を使います。ここで名前を詰め替えておくと画面やルーターは Cookie の都合を知らずに済みます。これで `src/lib/session.ts` は完成です。

`src/server/api/trpc.ts`

インポートとコンテキスト:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/trpc.ts
// 完成版: インポートとコンテキスト

import { initTRPC, TRPCError } from '@trpc/server';
import superjson from 'superjson';
import { ZodError } from 'zod';
import { USER_ROLE } from '@lib/constant/roles';
import { prisma } from '@lib/prisma';
import { getSession } from '@lib/session';

export const createTRPCContext = async (
  opts: { headers: Headers },
) => {
  const session = await getSession();

  return {
    session,
    ...opts,
  };
};

export type Context = Awaited<
  ReturnType<typeof createTRPCContext>
>;
```

`createTRPCContext` はリクエストが届くたびに1回だけ走ります。ここで `getSession` を呼んでおくと以降のすべての手続きが `ctx.session` から同じ結果を読めます。`Context` 型を `Awaited<ReturnType<...>>` で作ってあるので返す項目を増やしたときに型を手で書き直す必要はありません。

initTRPC の初期化:

```
// filepath: src/server/api/trpc.ts
// 完成版: initTRPC の初期化
const t = initTRPC.context<Context>().create({
  transformer: superjson,
  errorFormatter({ shape, error }) {
    return {
      ...shape,
      data: {
        ...shape.data,
        zodError:
          error.cause instanceof ZodError
            ? error.cause.flatten()
            : null,
      },
    };
  },
});
```

`transformer: superjson` を入れると `Date` や `Map` をそのままの型でサーバーとブラウザの間にやり取りできます。`errorFormatter` は zod の検証エラーを `zodError` として取り出す指定です。フォームの入力ミス、画面側が項目ごとに読み分けられる形へ整えて返します。

ログイン確認の前半:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/trpc.ts
// 完成版: ログイン確認の前半
const isAuthenticated = t.middleware(
  async ({ ctx, next }) => {
    if (!ctx.session?.userId) {
      throw new TRPCError({
        code: 'UNAUTHORIZED',
        message: 'ログインが必要です',
      });
    }

    const currentUser = await prisma.user.findUnique({
      where: { id: ctx.session.userId },
      select: {
        id: true,
        email: true,
        role: true,
        isActive: true,
        sessionVersion: true,
      },
    });
```

`t.middleware` は手続きの本体が走る前に挟み込む関門です。まず `ctx.session?.userId` が無ければ `UNAUTHORIZED` で止めます。通ったあとに DB を引き直しているのはトークンを発行したあとにアカウントが消されたり無効化されたりする場合があるからです。

ログイン確認の後半:

```
// filepath: src/server/api/trpc.ts
// 完成版: ログイン確認の後半(存在・有効化・版数チェック)

if (!currentUser) {
  throw new TRPCError({
    code: 'UNAUTHORIZED',
    message: 'ユーザーが見つかりません',
  });
}

if (!currentUser.isActive) {
  throw new TRPCError({
    code: 'FORBIDDEN',
    message: 'このアカウントは無効化されています',
  });
}

if (currentUser.sessionVersion !==
ctx.session.version) {
  throw new TRPCError({
    code: 'UNAUTHORIZED',
    message:
      'セッションが無効になりました。再度ログインしてください',
  });
}
```

ユーザーが見つからなければ `UNAUTHORIZED`、見つかったも `isActive` が偽なら `FORBIDDEN`、`sessionVersion` が不一致なら `UNAUTHORIZED` を返します。理由ごとにエラーを分けることで、失効を正しく検知できます。

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/trpc.ts
// 完成版: ログイン確認の後半(最新クレームの上書きと継続)

return next({
  ctx: {
    session: {
      ...ctx.session,
      email: currentUser.email,
      role: currentUser.role,
    },
  },
});
```

最後の `next` で、DB から取り直した最新の `email` と `role` を `ctx` に上書きして先へ渡します。

管理者チェックと4種類の入口:

```
// filepath: src/server/api/trpc.ts
// 完成版: 管理者チェックと4種類の入口
const isAdmin = t.middleware(async ({ ctx, next }) => {
  if (ctx.session?.role !== USER_ROLE.ADMIN) {
    throw new TRPCError({
      code: 'FORBIDDEN',
      message: '管理者権限が必要です',
    });
  }

  return next({ ctx });
});

export const createTRPCRouter = t.router;
export const publicProcedure = t.procedure;
export const protectedProcedure =
  t.procedure.use(isAuthenticated);
export const adminProcedure =
  t.procedure.use(isAuthenticated).use(isAdmin);
export const createCallerFactory = t.createCallerFactory;
```

`isAdmin` は役割だけを見る短い関門です。下の4行がこの先すべてのルーターが使う入口になります。`publicProcedure` は誰でも、`protectedProcedure` はログイン済みだけ、`adminProcedure` は管理者だけが通れます。手続きを書くときに入口を選ぶだけで、認証の判定が付いてきます。

`src/server/api/routers/_helpers/select.ts`

ファイル全体:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/routers/_helpers/select.ts
// 完成版: ファイル全体

import { z } from 'zod';
import { PROJECT_MEMBER_ROLE } from
'@/lib/constant/roles';

export const USER_SELECT = {
  id: true,
  name: true,
  email: true,
  avatar: true,
} as const;

export const USER_DETAIL_SELECT = {
  id: true,
  email: true,
  name: true,
  avatar: true,
  role: true,
  isActive: true,
} as const;

export const projectMemberRoleSchema =
  z.nativeEnum(PROJECT_MEMBER_ROLE);
```


Prisma に「どの列を返すか」を渡すための定型をまとめたファイルです。

`as const` を付けてあるのであとから中身を書き換えられず、Prisma が返す型もこの並びから決まります。パスワードの列が入っていないことを確かめてください。取得する列を毎回書かずに済むうえ、書き忘れて余計な列を返す事故も防げます。

`src/server/api/routers/auth.ts`

インポート:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/routers/auth.ts
// 完成版: インポート
import { TRPCError } from '@trpc/server';
import bcrypt from 'bcryptjs';
import { z } from 'zod';
import { USER_ROLE } from '@/lib/constant/roles';
import { createPasswordSchema } from '@/lib/password';
import { prisma } from '@/lib/prisma';
import {
  checkLoginRateLimit,
  extractClientIp,
  rateLimitToTRPCError,
  recordLoginSuccess,
} from '@/lib/rate-limit';
import { createSession, deleteSession, type SessionUser }
from '@/lib/session';
import { createTRPCRouter, protectedProcedure,
publicProcedure } from '../trpc';
import { USER_DETAIL_SELECT } from '../_helpers/select';
import { isUserEmailUniqueConstraintError } from
'../_helpers/user-email-conflict';
```

認証ルーターが借りてくるものの一覧です。`bcryptjs` はパスワードの照合、`@/lib/rate-limit` は連続失敗の制限、`@/lib/session` は先ほど作ったセッション管理です。最後の2行は1つ上のフォルダの `trpc.ts` と、同じフォルダの `_helpers/select.ts` から読み込みます。

入力スキーマ:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: 入力スキーマ

const loginSchema = z.object({
  email:
    z.string().email('有効なメールアドレスを入力してください'),
  password: z.string().min(1,
    'パスワードを入力してください'),
});

const registerSchema = z.object({
  name: z.string().min(1, '名前を入力してください'),
  email:
    z.string().email('有効なメールアドレスを入力してください'),
  password:
    createPasswordSchema('パスワードは8文字以上で入力してください'),
});
```

ログインは形だけ、登録は文字数と文字種まで見ます。ここで弾いた入力手続きの本体に届かないため `login` や `register` の中で入力の形を確かめる必要はありません。登録側の4本の `.regex()` はDay 06 の登録画面が持つ条件とそろえてあります。

想定外エラーの受け皿:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/routers/auth.ts
// 完成版: 想定外エラーの受け皿

function handleUnexpectedError(context: string, error:
unknown): never {
  console.error('[auth] unexpected error', { context,
error });

  throw new TRPCError({
    code: 'INTERNAL_SERVER_ERROR',
    message: `${
context}中にエラーが発生しました。時間をおいてお試しください。`,
    cause: error,
  });
}
```

`never` を返す型にしてあるのでこの関数を呼んだ先は「必ず投げて終わる」と TypeScript が判断します。中身をそのまま画面へ返さずログにだけ残すのは DB の接続情報のような内部の事情が利用者の画面に出るのを避けるためです。

login — 回数制限の確認:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: login - 回数制限の確認

export const authRouter = createTRPCRouter({
  login: publicProcedure.input(loginSchema).mutation(
    async ({ input, ctx }) => {
      const ip = extractClientIp(ctx.headers);

      // brute force 対策: 直近 15 分の失敗回数を
      // email 単独 / email×IP / IP 単独の3軸で rate-limit。

      // checkLoginRateLimit は許可と同時に失敗行を先取りで記録する
      // (成功時は recordLoginSuccess が削除)

      const limitResult = await checkLoginRateLimit(
        input.email, ip);
      if (!limitResult.allowed) {
        throw rateLimitToTRPCError(limitResult);
      }
    }
  )
});
```

ルーターの中で最初に走るのが回数制限の判定です。`checkLoginRateLimit` は許可を返すと同時に失敗の記録を先に1件置きます。成功したときだけ後で取り消す形にしてあるので途中で処理が止まっても失敗が数え漏れになりません。

login — ユーザーの取得:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/routers/auth.ts
// 完成版: login — ユーザーの取得

try {
  const user = await prisma.user.findUnique({
    where: { email: input.email },
  });

  if (!user?.password) {

// 失敗は checkLoginRateLimit が先取りで記録済みのため、
// ここでは追加記録しない
    throw new TRPCError({
      code: 'UNAUTHORIZED',
      message:
        'メールアドレスまたはパスワードが正しくありません',
    });
  }
}
```

`try` の中に入れてあるのはこの先で投げる `TRPCError` と、DB の障害のような想定外の例外を、最後の `catch` で仕分けるためです。ユーザーが見つからない場合とパスワードが未設定の場合を `!user?.password` の1本でまとめ、どちらも同じ文言で返します。

login — 照合と有効判定:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: login - 照合と有効判定

    const isPasswordValid = await bcrypt.compare(
input.password, user.password);
    if (!isPasswordValid) {

// 失敗は checkLoginRateLimit が先取りで記録済みのため、
// ここでは追加記録しない
    throw new TRPCError({
        code: 'UNAUTHORIZED',
        message:
'メールアドレスまたはパスワードが正しくありません',
    });
}

    if (!user.isActive) {
        // 無効判定はパスワード照合が通ったあとに行う。
        // 先に判定すると、正しいパスワードを知らなくても
        // 「無効化されたアカウントが存在する」ことを
        // 列挙できてしまうため。
        throw new TRPCError({
            code: 'FORBIDDEN',
            message: 'このアカウントは無効化されています',
        });
    }
}
```

Day 07: 認証バックエンドを作ってログインを動かそう

`bcrypt.compare` は保存されたハッシュと入力を突き合わせます。メールが違ふときとパスワードが違ふときで文言を変えていないのはどちらが間違っているかを外から探れないようにするためです。無効化の判定をパスワード照合のあとに置いている理由はコード中のコメントに書いたとおりです。

login — セッション発行と戻り値:


```
// filepath: src/server/api/routers/auth.ts
// 完成版: login - セッション発行と戻り値

const sessionUser: SessionUser = {
  id: user.id,
  email: user.email,
  role: user.role,
  version: user.sessionVersion,
};

// セッション発行の前に成功記録を確定させる。
// 順序が逆だと、記録の失敗で 500 を返す一方で
// 認証済みセッションだけが残るため。
await recordLoginSuccess(input.email, ip);
await createSession(sessionUser);

return {
  user: {
    id: user.id,
    email: user.email,
    name: user.name,
    avatar: user.avatar,
    role: user.role,
  },
};
```

Day 07: 認証バックエンドを作ってログインを動かそう

`recordLoginSuccess` を `createSession` より先に呼ぶ順番には理由があります。逆にすると記録に失敗して 500 を返ししながら、ログイン済みの Cookie だけが残る状態が起きます。戻り値からパスワードを外してあるので画面側が誤って表示する余地がありません。

login — 例外の仕分け:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: login — 例外の仕分け
    } catch (error) {
      if (error instanceof TRPCError) throw error;
      handleUnexpectedError('ログイン処理', error);
    }
  },
```

自分で投げた `TRPCError` はそのまま外へ通し、それ以外だけを `handleUnexpectedError` に渡します。この1行が無いと `UNAUTHORIZED` として投げたエラーまで 500 に化けて画面が「メールアドレスまたはパスワードが正しくありません」を出せなくなります。

register — 重複チェックとハッシュ化:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: register - 重複チェックとハッシュ化
register: publicProcedure
  .input(registerSchema)
  .mutation(async ({ input }) => {
    try {
      const existing = await prisma.user.findUnique({
        where: { email: input.email },
      });
      if (existing) {
        throw new TRPCError({
          code: 'CONFLICT',
          message:
            'このメールアドレスは既に登録されています',
        });
      }

      const hashedPassword = await bcrypt.hash(
        input.password,
        10,
      );
    }
  });
```

同じメールアドレスがすでにあれば **CONFLICT** で止めます。**bcrypt.hash** の第2引数 **10** は計算の重さを決める数値で、大きいほど総当たりで時間がかかる代わりにログインも遅くなります。生のパスワードはこの行から先へ持ち回りません。

register — 作成とセッション発行:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/routers/auth.ts
// 完成版: register — 作成とセッション発行

const user = await prisma.user.create({
  data: {
    email: input.email,
    name: input.name,
    password: hashedPassword,
    role: USER_ROLE.USER,
    isActive: true,
  },
});

const sessionUser: SessionUser = {
  id: user.id,
  email: user.email,
  role: user.role,
  version: user.sessionVersion,
};

await createSession(sessionUser);
```

`role` に `USER_ROLE.USER` を固定で入れているのは入力から役割を受け取らないためです。ここを入力任せにすると登録の申し込みに `role: 'ADMIN'` を混ぜるだけで管理者になれてしまいます。作成できたらログインと同じ `createSession` を呼び、そのまま使い始められる状態にします。

register — 戻り値と例外:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: register - 戻り値と例外

    return {
      user: {
        id: user.id,
        email: user.email,
        name: user.name,
        avatar: user.avatar,
        role: user.role,
      },
    };
  } catch (error) {
    if (error instanceof TRPCError) throw error;
    if (isUserEmailUniqueConstraintError(error)) {
      throw new TRPCError({
        code: 'CONFLICT',
        message:
          'このメールアドレスは既に登録されています',
      });
    }
    handleUnexpectedError('ユーザー登録処理', error);
  }
},
```

返す項目はログインとそろえてあります。画面側は登録直後とログイン直後で同じ形のデータを受け取れます。`catch` はログインと共通ですが、DBが返した `User.email` の一意制約エラーは事前確認と同じ `CONFLICT` へ変換します。他の想定外エラーだけを `handleUnexpectedError` に渡します。

Day 07: 認証バックエンドを作ってログインを動かそう

logout:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: logout
logout: publicProcedure.mutation(async () => {
  await deleteSession();
  return { success: true };
}),
```

logout は現在のブラウザの Cookie を消してログアウトします。サーバー側にセッション一覧を保存していないのでDBレコードは消しません。また、別端末の Cookie まで直接削除する手続きではありません。

getSession:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: getSession

getSession: publicProcedure.query(async ({ ctx }) => {
  if (!ctx.session) return null;

  const user = await prisma.user.findUnique({
    where: { id: ctx.session.userId },
    select: { ...USER_DETAIL_SELECT, sessionVersion:
true },
  });

  if (!user || !user.isActive ||
user.sessionVersion !== ctx.session.version) {
    return null;
  }

  return {
    user: {
      id: user.id,
      email: user.email,
      name: user.name,
      avatar: user.avatar,
      role: user.role,
      isActive: user.isActive,
    },
  };
}),
```

Day 07: 認証バックエンドを作ってログインを動かそう

`logout` は Cookie を消すだけで終わります。`getSession` を `publicProcedure` にしてあるのは未ログインの画面からも呼ばれるからです。ログインしていなければエラーではなく `null` を返し、画面は「ログインしていない状態」として描き分けます。また、DB の `isActive` や `sessionVersion` が一致しない無効なトークンに対しても `null` を返します。

getCurrentUser — 取得:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: getCurrentUser — 取得

getCurrentUser: protectedProcedure.query(
  async ({ ctx }) => {
    const user = await prisma.user.findUnique({
      where: { id: ctx.session.userId },
      select: {
        ...USER_DETAIL_SELECT,
        createdAt: true,
        updatedAt: true,
      },
    });
    if (!user) {
      throw new TRPCError({
        code: 'NOT_FOUND',
        message: 'ユーザーが見つかりません',
      });
    }
  }
);
```


こちらは `protectedProcedure` なので関門を通った時点で `ctx.session` が存在すると確定しています。`USER_DETAIL_SELECT` に作成日時と更新日時を足して取り出し、プロフィール画面で使える形にします。

`getCurrentUser` — 判定と締め:

```
// filepath: src/server/api/routers/auth.ts
// 完成版: getCurrentUser - 判定と締め

    if (!user.isActive) {
      throw new TRPCError({
        code: 'FORBIDDEN',
        message: 'このアカウントは無効化されています',
      });
    }

    return user;
  },
),
});
```

無効化されたアカウントは `FORBIDDEN` で弾きます。最後の `});` がルーター全体の閉じ括弧です。この行より下にコードを足すとルーターの外に出てしまい、英語のエラーで止まります。

`src/server/api/root.ts`

ファイル全体:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/server/api/root.ts
// 完成版: ファイル全体
import { authRouter } from './routers/auth';
import { createCallerFactory, createTRPCRouter } from
'./trpc';

export const appRouter = createTRPCRouter({
  auth: authRouter,
});

export type AppRouter = typeof appRouter;

export const createCaller = createCallerFactory(
appRouter);
```

アプリが持つ手続きの一覧表です。今日の時点では `auth` の1本だけを載せます。
`AppRouter` 型を書き出しているのでブラウザ側は `api.auth.login` と打った
時点で入力と戻り値の型を受け取れます。Day 09 以降でルーターを足すときはここ
に1行ずつ加えていきます。

`src/app/api/trpc/[trpc]/route.ts`

ファイル全体:

```
// filepath: src/app/api/trpc/[trpc]/route.ts
// 完成版: ファイル全体
import { fetchRequestHandler } from
  '@trpc/server/adapters/fetch';
import type { NextRequest } from 'next/server';
import { appRouter } from '@server/api/root';
import { createTRPCContext } from '@server/api/trpc';

const handler = (req: NextRequest) =>
  fetchRequestHandler({
    endpoint: '/api/trpc',
    req,
    router: appRouter,
    createContext: () =>
      createTRPCContext({ headers: req.headers }),
  });

export { handler as GET, handler as POST };
```

ブラウザからの HTTP リクエストを tRPC に橋渡しする入口です。`endpoint` に書いた `/api/trpc` はフォルダ名 `src/app/api/trpc/[trpc]` と対応しています。`createContext` を関数のまま渡してあるのはリクエストが届くたびに新しいコンテキストを作るためです。

`src/middleware.ts`

インポートと公開パス:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/middleware.ts
// 完成版: インポートと公開パス
import { jwtVerify } from 'jose/jwt/verify';
import { type NextRequest, NextResponse } from
'next/server';
import { isValidRedirectUrl } from '@lib/redirect';

const COOKIE_NAME = 'session';

const PUBLIC_PATHS = ['/login', '/register'];
function isPublicPath(pathname: string): boolean {
  return PUBLIC_PATHS.some(
    (path) =>
      pathname === path
      || pathname.startsWith(`${path}/`),
  );
}
```

`session.ts` ではなく `jose` を直接読み込んでいます。`middleware` は Edge Runtime という軽い実行環境で動き、そこでは `cookies()` を使う `session.ts` を読み込めないためです。`isPublicPath` は `/login` と `/register`、およびその配下を公開扱いにします。

戻り先の検証と鍵:

```
// filepath: src/middleware.ts
// 完成版: 戻り先の検証と鍵

function isValidCallbackPath(path: string): boolean {
  return (
    isValidRedirectUrl(path)
    && !path.includes('/://')
  );
}

function getJwtSecret(): Uint8Array {
  const secret = process.env['JWT_SECRET'];
  if (!secret) {
    throw new Error('JWT_SECRET is not set');
  }
  return new TextEncoder().encode(secret);
}
```

`isValidCallbackPath` はログイン後の戻り先として渡された文字列が自分のサイト内かを確認めます。`://` を弾いているのは `https://悪意のあるサイト` を戻り先に仕込まれる経路を塞ぐためです。鍵の取り出しでは `process.env` を直接読みます。Edge Runtime では `@/lib/env` を使えません。

素通りさせる2つの経路:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/middleware.ts
// 完成版: 素通りさせる2つの経路
export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // 公開ページはスキップ
  if (isPublicPath(pathname)) {
    return NextResponse.next();
  }

  // tRPC エンドポイントはスキップ
  // (tRPC 層の protectedProcedure で認証を担保)
  if (pathname.startsWith('/api/trpc')) {
    return NextResponse.next();
  }
}
```

判定を上から順に並べ、当てはまったらその場で返します。公開ページと `/api/trpc` はここで抜けます。tRPC を素通りさせても穴にはなりません。`protectedProcedure` が同じ判定をサーバー側で行うためです。

Cookie が無い場合:

```
// filepath: src/middleware.ts
// 完成版: Cookie が無い場合

const token = request.cookies.get(COOKIE_NAME)?.value;

// Cookie なし → ログインへリダイレクト
if (!token) {
  const loginUrl = new URL('/login', request.url);
  loginUrl.searchParams.set(
    'callbackUrl',
    isValidCallbackPath(pathname)
      ? pathname
      : '/dashboard',
  );
  return NextResponse.redirect(loginUrl);
}
```

Cookie が無ければログイン画面へ送ります。このとき `callbackUrl` に元のページを付けておくとログイン後にそこへ戻せます。検証を通らなかった場合は `/dashboard` に落とすので外部のアドレスが戻り先に紛れ込みません。

JWT の検証:

Day 07: 認証バックエンドを作ってログインを動かそう

```
// filepath: src/middleware.ts
// 完成版: JWT の検証
// JWT 検証
const jwtSecret = getJwtSecret();
try {
  await jwtVerify(token, jwtSecret, {
    algorithms: ['HS256'],
  });
  return NextResponse.next();
} catch {
  // 無効なトークン: Cookie 削除してログインへ
  const loginUrl = new URL('/login', request.url);
  const response = NextResponse.redirect(loginUrl);
  response.cookies.delete(COOKIE_NAME);
  return response;
}
```

署名が合えば `NextResponse.next()` でそのまま先へ通します。合わなければ Cookie を消してからログイン画面へ送ります。消しておかないと壊れたトークンを持ったまま毎回リダイレクトが起き、利用者から見ると画面が往復し続けます。

適用範囲の指定:


```
// filepath: src/middleware.ts
// 完成版: 適用範囲の指定
export const config = {
  matcher: [
    '/((?!_next/static|_next/image|favicon.ico).*)',
  ],
};
```

`matcher` は middleware を走らせる範囲の指定です。`_next/static` や `favicon.ico` を外してあるのは画像やスクリプトを読み込むたびに JWT を検証すると無駄が大きいためです。

ここで挙動が1つ変わります。この書き方は「除外したもの以外すべて」なのでトップページの `/` も middleware の対象に入ります。`/` は `PUBLIC_PATHS` に入っていないため今日からはログインしていない状態で `/` を開くと `/login` へ送られます。Day 01 で作ったあの画面はログイン後でないと見られなくなります。壊れたわけではありません。

ログイン前でも見せたいなら `PUBLIC_PATHS` に `'/'` を足します。

`isPublicPath` の前方一致は「そのパスの後ろに `/` を付けた文字列で始まるか」を見るので `'/'` に対しては `//` で始まるかの判定になります。`/dashboard` などとは当てはまりません。つまり `'/'` を足しても公開になるのは `/` だけで、他の画面は保護されたままです。このカリキュラムでは `/` をログイン後の入口として扱うので足さずに進みます。

これで `src/middleware.ts` は完成です。

完成形の参考コード: 完成版のリポジトリにも、今日書き換えた9つのファイルと確認した `sessionVersion` の列があります。ただし今日書いたコードと1文字まで同じではありません。主な違いを例示します。完成版の `trpc.ts` と `middleware.ts` には、障害を追いかけるためのリクエスト ID とログ出力があります。`root.ts` には `project` や `task` を含む7本のルーターがありますが、今日の時点では `auth` の1本だけです。`session.ts` では、今日はCookieの保存を `saveSessionCookie` に分け、完成版は `createSession` の中で直接保存します。`auth.ts` の利用者判定も、今日は `!user || !user.isActive`、完成版は `!user?.isActive` と書きます。最後の2つは処理の分け方や条件の書き方が違うだけで、どちらもCookieを保存し、利用者が見つからない場合と無効な場合に `null` を返します。このほかにもコメントや行の分け方に違いがあります。Day 09 以降でルーターを1本ずつ増やします。(販売用 ZIP に完成版の `src/` は入っていません。ここに挙げた違いは完成版がどう書かれているかの説明として読んでください)

完成コードを書き終えたらターミナルで `npm run type-check` を実行してください。型のエラーが出た場合は表示されたファイル名と行番号を確認して直します。画面の動作は本文の操作手順で別に確かめます。

今日のまとめ

- ☐ `prisma/schema.prisma` の `sessionVersion` が1本だけあることを確認しました。
- ☐ `src/lib/password.ts` と `user-email-conflict.ts` を全文で書き直しました。
- ☐ `src/lib/session.ts` — JWT セッション管理を作り直しました。

- ☐ `src/server/api/trpc.ts` — tRPC の土台を作り直しました。
- ☐ `src/server/api/routers/auth.ts` — 認証ルーターを作り直しました。
- ☐ `src/server/api/routers/_helpers/select.ts` — 返すユーザー情報の項目を定義しました。
- ☐ `src/server/api/root.ts` + `route.ts` — API を繋ぎ直しました。
- ☐ `src/middleware.ts` — ルート保護を作りました。
- ☐ ログインが実際に動くことを確認しました。
- ☐ DevTools で JWT と Cookie の中身を確認しました。

つまずきポイント

`JWT_SECRET is not set`

原因

実行中の環境に `JWT_SECRET` が設定されていません。ローカルの `.env` と公開先の Vercel は別々に設定します。

解決方法

ローカルでは `.env` を確認し、値が無ければ32文字以上の鍵を設定して開発サーバーを再起動します。公開先では Vercel の Environment Variables に `JWT_SECRET` を設定し、再デプロイしてから開き直します。Day 04 で説明した環境変数の設定先を確認してください。

`Cannot find module '@/lib/session'`

原因

Day 07: 認証バックエンドを作ってログインを動かそう

ファイルパスの綴りが違います。

解決方法

`src/lib/session.ts` にあるか確認します。

`Cannot find module '@server/api/root'`

原因

`root.ts` を作成していません。

解決方法

Step 4 の `root.ts` を作成します。

ログインしてもトーストが出ない

原因

`auth` ルーターが `root.ts` に登録されていません。

解決方法

`root.ts` に `auth: authRouter` があるか確認します。

`UNAUTHORIZED: ログインが必要です`

原因

`Cookie` が保存されていません。

解決方法

DevTools → Application → Cookies で `session` を確認します。

`prisma.user.findUnique is not a function`

原因

Prisma Client が生成されていません。

解決方法

`npx prisma generate` を実行します。

`The table `public.users` does not exist in the current database.``

原因

DB にテーブルがありません。

解決方法

`npm run db:push && npm run db:seed` を実行します。

`ログイン試行回数が上限に達しました`

原因

同じメールを同じ回線から 5 回、同じメールで回線を変えながら 10 回、または同じ回線から合計 20 回失敗したため、一時的にロックされています。

解決方法

Day 07: 認証バックエンドを作ってログインを動かそう

15分待ちます。メールと回線は変えずに待つのがいちばん早い方法です。コードの問題ではありません。

middleware.ts が効かない

原因

ファイルの置き場所が違っています。

解決方法

`src/middleware.ts` に置きます。`src/app/` ではなく、`src/` の直下です。

今日学んだ用語

用語	意味
JWT	利用者の情報に署名を付けた文字列。 中身は読めるが書き換えると署名が合わなくなる
署名	秘密の鍵で作る封印。中身が途中で書き換えられていないかを確認するために使う
exp	JWT の中に書く有効期限。秒で数える
bcrypt	パスワードを元に戻せない形へ変換する仕組み
HttpOnly Cookie	JavaScript から読めない Cookie。 盗み見の被害を狭める

用語	意味
maxAge	Cookie 自体の寿命。ブラウザ側が持つので JWT の <code>exp</code> とは別に必要
publicProcedure	ログインしていなくても呼べる tRPC の入口
protectedProcedure	ログイン済みの人だけが呼べる tRPC の入口
adminProcedure	管理者だけが呼べる tRPC の入口
middleware	ページが表示される前に毎回動く処理。 ログインの有無で行き先を振り分ける
matcher	middleware を動かす URL を絞り込む指定
rate limit	短時間に失敗が続いたとき一時的に受け付けを止める仕組み

応用課題: 認証失敗の表示を 2 通りで確かめる

登録済みかどうかをエラー文から区別できないことを確かめましょう。利用者を探す処理とパスワード照合は別でも、表示はそろえます。

(1) `/login` を開き、未登録の `practice-missing@example.invalid` と空でない任意のパスワードでログインを試します。表示されたエラー文をメモします。

(2) 次に今日ログインできたメールアドレスと、それとは違うパスワードで試します。メールアドレスの入力形式は変えません。

(3)2回とも同じエラー文でログイン画面に残れば成功です。違う文が出た場合は

`src/server/api/routers/auth.ts` の `login` にある利用者不在とパスワード不一致の分岐を読み直します。

(4)正しいメールアドレスとパスワードでログインし、ダッシュボードへ戻ります。認証処理のコードは変更しません。

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1. `isAuthenticated` の中で `prisma.user.findUnique` を引き直し、`role: currentUser.role` で上書きしているのは何をしている処理ですか。

A. JWT に入っている `role` はログインした瞬間の値のまま固定されています。7日間そのままです。そこでリクエストのたびにデータベースから今の権限を取り直し、古い値を差し替えています。この1行が無いと管理者権限を外された相手がトークンの期限が切れるまで管理者として動けます。

Q2. `createSession` の `Math.floor(Date.now() / 1000)` から `/ 1000` を消すと何が起きますか。

A. トークンの期限は変わりません。`signSessionToken` の `.setExpirationTime('7d')` が `exp` を7日後で上書きするので `createSession` で計算した値はトークンには残らないためです。Cookie の寿命も `maxAge: COOKIE_MAX_AGE` という定数で決まっていてこの計算とは別です。`/ 1000` を消すと `SessionPayload` が持つ数字だけが1000倍ずれます。期限を判断しているのがどの1行なのかを、ここで確かめてください。

Q3. `login` で「そのメールアドレスは存在しない」と「パスワードが違う」を、同じ文
言で返しているのはなぜですか。

A. 区別して返すとどのメールアドレスが登録済みかを外から調べられるからです。総
当たりで問い合わせれば利用者の一覧を作れてしまいます。同じ理由で、`isActive`
の判定もパスワードの照合より後ろに置いています。判定の順番そのものが守りの一
部になっています。

次回予告

Day 08 ではサイドバー付きのアプリレイアウトを作ります。ログアウトボタン、ユー
ザー情報ウィジェット、ナビゲーション——今日作った認証バックエンドの上に、使い
やすい UI を組み立てていきます。

次に読むもの

- 前の日: Day 06^{*1}
- 次の日: Day 08^{*2}
- 全体の地図: 学びのロードマップ^{*3}
- 目次: カリキュラム目次^{*4}
- 詰まったとき: トラブルシューティング^{*5}

1. https://drive.google.com/file/d/1LeLRhUPR3F9sdG_1FVHD8QSXyiHTfUf6/view?usp=drivesdk

2. https://drive.google.com/file/d/1Oj1x_743DXVEeRUK7SgLdRZ9LGe6O6Y/view?usp=drivesdk

3. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk

4. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>

5. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>

- 言葉の意味: 用語集^{*6}

奥付

タイトル	Day 07: 認証バックエンドを作ってログインを動かそう
著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。

6. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>